Empirica: Epistemic Self-Assessment for AI Systems

Measuring Knowledge State Through Functional Metacognition

David Samuel L. Van Assche
Empirica
david@getempirica.com

March 2, 2026

Abstract

We present Empirica, a framework for AI epistemic self-assessment that operationalizes the principle that AI systems must investigate before acting. We report production data from a living system that calibrates through its own operation—not a static benchmark. Through 1,190,265 evidence observations across 792 sessions, we demonstrate that effective AI capability—model weights plus externalized epistemic scaffolding—genuinely increases during task execution. The learning delta is real, not an artifact of assessment correction: model weights remain static while the external knowledge infrastructure grows. Filtering to 456 clean learning pairs (excluding default assessments and incomplete workflows), 83.3% showed knowledge improvement with a mean effective capability growth of +0.162. We introduce a 13-dimensional epistemic vector representation capturing what AI systems know, what they can do, and how ready they are to act. Bayesian calibration analysis reveals unified confidence growth across all vectors during learning, with calibration variance dropping 107× as evidence accumulates. The framework implements epistemic gates that enforce investigation when knowledge is insufficient, preventing the "overconfidence trap" where AI systems act on incomplete knowledge. We further introduce dual-track calibration: self-referential corrections from learning deltas alongside grounded corrections from post-test evidence (test results, git metrics, goal completion), partially addressing the open problem of externally validating AI self-assessment. Grounded verification now covers 1,190 beliefs with 288 post-test verifications across 9 vectors. We release Empirica as open-source infrastructure for epistemic gating, calibration tracking, and structured AI workflows.

Keywords: AI self-assessment, epistemic vectors, metacognition, calibration, uncertainty quantification

1 Introduction

1.1 The Execution-Layer Fallacy

The dominant paradigm for deploying AI systems treats them as execution layers: a user provides a task, the AI executes it, and returns output. This model—borrowed from traditional software engineering—assumes capability is fixed and known. Give the system an input; receive a deterministic output.

This assumption fails catastrophically for large language models.

When an LLM receives a complex task, it does not possess fixed, queryable knowledge. Its effective capability on *that specific task* depends on factors that emerge only through engagement: what context it accumulates, what connections it discovers, what it learns by exploring the problem space. An LLM asked to "fix the authentication bug" may have latent capability to solve it—but that capability becomes *actual* only through investigation.

The execution-layer model ignores this. It assumes the AI either can or cannot perform the task, and proceeds accordingly. The result is a familiar failure mode:

- AI reports low uncertainty (it "thinks" it knows)
- AI has low actual capability (it doesn't actually know)
- AI proceeds without investigation
- AI produces confident, wrong output
- User loses trust

We call this the **overconfidence trap**: the AI's self-assessed certainty exceeds its actual capability, leading to premature action without adequate exploration.

1.2 Noetic Before Praxic

This paper proposes a different model: **AI systems must investigate before acting**. We formalize this as two phases of AI cognition:

Noetic phase: Exploration, hypothesis formation, knowledge acquisition. The AI investigates the problem, reads relevant context, identifies unknowns, and builds genuine understanding.

Praxic phase: Execution, implementation, action. The AI applies its acquired knowledge to produce output.

The critical insight is that these phases are not optional or interchangeable. *Noetic must precede praxic.* An AI that acts without first investigating is an AI that acts on incomplete knowledge—and incomplete knowledge produces errors.

Put differently: this is about **interpretation quality**, not raw capability. The same model, given the same prompt, will produce dramatically different outputs depending on whether it acts immediately on its initial interpretation or first investigates to build a grounded understanding. One-shot prompting assumes the AI's immediate interpretation is sufficient. Noetic-first recognizes that complex tasks require the AI to validate its interpretation—to check what it thinks the user wants against what the codebase actually contains, what the context actually implies, what the task actually requires. The 13 epistemic vectors we introduce measure how complete and grounded this interpretation is, not how "smart" the AI is.

This is not a novel observation to practitioners who work with AI systems daily. It is, however, surprisingly absent from the research literature, which tends to evaluate AI on fixed benchmarks rather than dynamic, exploratory tasks.

1.3 The Learning Delta

If noetic precedes praxic—if investigation precedes action—then a measurable phenomenon should emerge: **effective AI capability should increase during task execution**. Not because model weights change, but because the external scaffolding—structured findings, resolved unknowns, calibrated corrections—accumulates knowledge that compounds the model's static reasoning.

An AI that begins a task with partial knowledge, investigates, and builds understanding will assess its own epistemic state differently at task-end than at task-start. The difference between these assessments—the *learning delta*—measures genuine knowledge acquisition externalized through epistemic artifacts.

We present evidence from 1,190,265 observations across 792 sessions showing exactly this pattern. Across all 13 epistemic dimensions we track, AI systems show systematic confidence growth during task execution:

Phase	Mean Capability Assessment
Pre-task (PREFLIGHT)	0.647
Post-task (POSTFLIGHT)	0.809
Learning Delta	**+0.162**

Table 1: Learning delta across 456 clean learning pairs (mean of 12 capability vectors).

Filtering to 456 clean learning pairs (excluding default assessments and incomplete workflows), **83.3% showed knowledge improvement**. This is not calibration error being corrected. This is learning being measured.

1.4 Empirica: Epistemic Infrastructure for AI

We introduce **Empirica**, a framework that operationalizes noetic-before-praxic through four mechanisms:

1. **Epistemic Vectors**: A 13-dimensional representation of AI self-assessed cognitive state organized into four tiers. *Foundation Knowledge* (KNOW, DO, CONTEXT) captures what the AI knows, what it can execute, and what environment it operates in. *Comprehension* (CLARITY, COHERENCE, SIGNAL, DENSITY) captures how well it understands the task. *Execution Readiness* (STATE, CHANGE, COMPLETION, IMPACT) captures how prepared it is to act. A *Meta-Layer* (ENGAGEMENT, UNCERTAINTY) captures collaborative quality and awareness of limitations. Together, these tiers measure the completeness and quality of the AI's task-specific interpretation—not raw capability, but grounded understanding.

2. **Epistemic Gates**: Decision points that evaluate the full vector space to determine whether the AI has sufficient knowledge to proceed. Gates enforce investigation when knowledge is incomplete, preventing premature action. The gate is calibration-aware: learned bias corrections adjust raw self-assessments before evaluation.

3. **The CASCADE Workflow**: A 4-phase loop—PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST—that captures epistemic state before and after task execution, measures learning across all vectors, and calibrates future assessments through dual-track Bayesian updates.

4. **Grounded Verification**: Post-test evidence collection that compares self-assessed epistemic state against objective outcomes (test results, git metrics, goal completion), anchoring calibration in what actually happened rather than further self-report. Currently 9 of 13 vectors are grounded in objective evidence.

Together, these mechanisms transform AI from an execution layer into a **learning system**—one that investigates before acting, measures its own knowledge acquisition across all epistemic dimensions, and improves through feedback grounded in objective outcomes.

1.5 Contributions

This paper makes the following contributions:

1. **Conceptual**: We articulate the noetic-before-praxic principle and argue that treating AI as an execution layer is a category error that produces systematic failures.

2. **Empirical**: We present 1,190,265 observations demonstrating that effective AI capability genuinely increases during task execution—not by modifying model weights, but through externalized epistemic scaffolding comprising seven interlocking processes: contextual injection, structured investigation, a cognitive immune system, persistent procedural knowledge, epistemic intent tracking, grounded verification, and compaction recovery. 83.3% of clean learning pairs show knowledge improvement.

3. **Practical**: We introduce Empirica, an open-source framework implementing epistemic gates, a 7-layer artifact taxonomy, four-layer memory architecture (HOT/WARM/SEARCH/-COLD), semantic retrieval, procedural knowledge (lessons), and dual-track calibration. The framework is production-tested across 792 sessions. Importantly, this is a *living system that calibrates through its own operation*—the data we report reflects a production calibration loop, not a static benchmark.

4. **Theoretical**: We develop a computational epistemology for AI systems—a formal theory of how information-processing agents can assess and manage their own knowledge state.

1.6 Paper Structure

Section 2 reviews related work in AI self-assessment, metacognition, and calibration. Section 3 presents the theoretical framework: epistemic vectors, their grounding in transformer attention mechanisms, and the noetic-praxic distinction. Section 4 describes the Empirica implementation. Section 5 presents empirical results, including the learning delta analysis. Section 6 describes the Sentinel protocol for epistemic gating. Section 7 discusses implications for AI deployment and alignment. Section 8 concludes.

The core claim of this paper is simple: AI systems that investigate before acting outperform AI systems that act without investigating. The evidence is in the delta.

2 Related Work

2.1 AI Calibration and Confidence

The problem of AI calibration—whether model confidence scores reflect actual accuracy—has received substantial attention. Guo et al. [2017] demonstrated that modern neural networks are poorly calibrated, often overconfident in incorrect predictions. Temperature scaling and other post-hoc calibration methods address this for classification tasks.

For language models, calibration becomes more complex. Kadavath et al. [2022] showed that large language models can be trained to express calibrated uncertainty through verbalized confidence. However, this work focuses on *reportive* confidence—telling humans how certain the model is—rather than *behavioral* confidence that routes the model's own decision-making.

Our work differs in treating self-assessment as a routing signal for the AI's own decision-making, not a report to external observers. The calibration we measure is between self-assessed epistemic state and subsequent task outcomes, not between verbalized confidence and answer correctness.

2.2 Metacognition in AI Systems

Metacognition—thinking about thinking—has been explored in AI through several lenses. Fleming and Dolan [2012] characterize metacognition as the ability to monitor and control cognitive processes. In AI, this translates to systems that can assess their own capabilities and adjust behavior accordingly.

Recent work on "self-reflection" in language models [Shinn et al., 2023, Madaan et al., 2023] prompts models to critique and revise their outputs. These approaches treat reflection as an output-improvement mechanism rather than an epistemic assessment framework.

The epistemic vector approach differs by making self-assessment *structural*—a 13-dimensional representation that persists across interactions and enables systematic calibration. Rather than ad-hoc reflection prompts, we propose a formal theory of what AI systems should assess about their own cognitive state.

2.3 Uncertainty Quantification

Uncertainty in neural networks has been extensively studied. Bayesian deep learning [Gal and Ghahramani, 2016] provides principled uncertainty estimates through dropout approximations. Ensemble methods [Lakshminarayanan et al., 2017] capture model uncertainty through prediction variance.

For language models, uncertainty quantification faces unique challenges. Token-level probabilities don't straightforwardly translate to task-level confidence. Kuhn et al. [2023] propose semantic uncertainty measures based on meaning equivalence of generated responses.

Our framework complements these approaches by operating at a higher level of abstraction. Rather than quantifying uncertainty in model weights or outputs, we track self-assessed uncertainty as one dimension of a broader epistemic state. The empirical finding that AI systems systematically overestimate uncertainty (Section 5) suggests that training processes may induce conservative biases beyond what principled uncertainty methods would produce.

2.4 RLHF and Alignment Training

Reinforcement Learning from Human Feedback (RLHF) has become standard for aligning language models with human preferences [Ouyang et al., 2022]. Constitutional AI [Bai et al., 2022] extends this with self-critique mechanisms.

A less-discussed effect of alignment training is its impact on model confidence. RLHF specifically targets overconfidence and hallucination, training models to express appropriate uncertainty. Our calibration data suggests this may have overcorrected: models trained to be humble may underestimate their actual capabilities across multiple dimensions.

This is not a criticism of RLHF—reducing harmful overconfidence is valuable. Rather, it suggests that alignment training produces systematic biases that can be measured and corrected. The learning delta we observe ($+0.162$ mean capability growth during tasks, with 83.3% of learning pairs showing improvement) may partially reflect AI systems overcoming training-induced conservatism through task-specific evidence.

A deeper structural issue: RLHF trains models to predict what evaluators want to hear, not to accurately report internal state. The reward signal comes from human preferences, not from epistemic accuracy. This creates an incentive to *appear* calibrated rather than *be* calibrated. Our framework addresses this by making self-assessment *functional*: the AI's self-report directly determines its subsequent behavior (proceed vs. investigate), and outcomes are measured through dual-track calibration. The grounded verification track (Section 5) compares self-assessment against objective evidence, detecting precisely the gap between reported and actual capability that RLHF incentives might create.

2.5 Human-AI Collaboration

Research on human-AI collaboration has explored how humans and AI systems can work together effectively. Bansal et al. [2021] study when humans should defer to AI recommendations. Lai et al. [2021] examine how AI explanations affect human decision-making.

This work typically positions AI as an advisor or tool that humans consult. Our framework inverts this relationship: the AI assesses its own epistemic state to decide whether to act or investigate, with the human providing task specification rather than real-time oversight.

The ENGAGEMENT vector in our framework captures collaborative quality—whether human and AI are aligned in understanding and goals. Low engagement triggers a stop condition, recognizing that productive collaboration requires mutual comprehension.

2.6 Planning and Reasoning in LLMs

Recent work on LLM reasoning [Wei et al., 2022, Yao et al., 2023a] demonstrates that chain-of-thought prompting and tree-of-thought exploration can improve model performance on complex tasks. These methods implicitly invoke investigation before action.

ReAct [Yao et al., 2023b] interleaves reasoning and acting, allowing models to gather information before committing to actions. This aligns with our noetic-before-praxic principle, though ReAct frames it as an emergent prompting strategy rather than a formal epistemic requirement.

Our contribution is to make this pattern explicit and measurable. The CASCADE workflow operationalizes investigation-before-action as a gated loop with quantified epistemic state. The learning delta provides empirical evidence that the pattern produces genuine capability growth, not just more cautious outputs.

2.7 Epistemic Measurement in AI

A question for any AI self-assessment framework is whether the measurements track something real. Recent work on emergent introspective awareness [Bricken and Anthropic, 2025] finds that models possess "a limited, functional form of introspective awareness" but that "abilities observed are highly unreliable, and failures of introspection remain the norm." Steyvers and Peters [2025] note that LLM metacognitive abilities are "limited in resolution, emerge in context-dependent manners, and seem qualitatively different from humans."

Our framework is compatible with these findings because it does not require reliable introspection. We measure information state: what the system knows relative to a task, what it can do, how complete its context is. These are structured prompts that produce numeric outputs—the same mechanism as when an AI estimates confidence, verifies facts, or scores relevance. The outputs are useful when they correlate with task-relevant states and calibrate over time.

The empirical evidence supports this: systematic calibration patterns across 1,190,265 observations (Section 5), 107× variance reduction with evidence accumulation, and grounded verification showing that self-assessed vectors track objective outcomes for capability dimensions. We don't assume perfect measurement—we measure systematic biases and correct for them through dual-track calibration. What these measurements reveal about the system's internal processes is a separate question this paper does not address.

2.8 Summary

Our work builds on and differs from prior research in several ways:

The novel contribution is the combination: a formal theory of AI epistemic state (Section 3), implemented as a practical framework (Section 4), validated with substantial empirical evidence (Section 5).

3 Theoretical Framework

3.1 Epistemic Vectors: A Computational Theory of Knowing

We propose that AI cognitive state can be represented as a vector in a 13-dimensional epistemic space. Each dimension corresponds to a measurable aspect of the system's knowledge state,

Approach	Focus	Our Contribution
Calibration literature	Confidence in predictions	Confidence in epistemic state
Metacognition research	Output reflection	Structural self-assessment
Uncertainty quantification	Model uncertainty	Self-assessed uncertainty
RLHF / Alignment	Reducing overconfidence	Measuring overcorrection
Human-AI collaboration	Human oversight	AI self-governance
LLM reasoning	Prompting strategies	Formal epistemic framework

Table 2: Comparison with prior approaches.

collectively capturing what the AI knows, what it can do, and how confident it should be in proceeding.

3.1.1 The Vector Taxonomy

The 13 epistemic vectors are organized into four tiers:

Tier 0: Meta-Cognition

ENGAGEMENT Quality of collaborative intelligence between human and AI. Functions as the primary gate: if engagement is below threshold, other assessments are unreliable.

Tier 1: Foundation Knowledge (35% weight in confidence calculation)

KNOW Domain knowledge and conceptual understanding

DO Technical execution capability and tool access

CONTEXT Environmental and systemic awareness

Tier 2: Comprehension (25% weight)

CLARITY Request specificity and goal definition

COHERENCE Logical consistency of approach

SIGNAL Useful information vs noise ratio

DENSITY Information packing efficiency (inverted: lower is better)

Tier 3: Execution Readiness (25% weight)

STATE Current state understanding

CHANGE Progress monitoring and regression detection

COMPLETION Path visibility to success

IMPACT Consequence and risk prediction

Meta-Layer (not in confidence calculation)

UNCERTAINTY Awareness of limitations and unknowns

3.1.2 Measurement with Behavioral Consequences

A critical design principle: these vectors are *behavioral* measurements, not reports. The distinction matters for calibration:

Reportive measurement: AI tells a human "I am 80% confident." The human must trust the report; the AI has no stake in accuracy.

Behavioral measurement: AI assesses its epistemic state to decide whether to proceed with a task. Bad assessment leads to bad outcomes, creating a feedback loop that enables calibration.

The epistemic vectors are routing signals for the AI's own decision-making. When an AI assesses KNOW = 0.4 and decides to investigate rather than proceed, the assessment directly affects behavior and outcomes. This is information-state measurement—analogous to how a search engine estimates result relevance, or a PID controller measures error—where the measurement has consequences that create a correction signal.

3.1.3 Confidence Calculation

Overall epistemic confidence aggregates the vectors:

$$\begin{aligned}\text{confidence} = {} & \text{ENGAGEMENT} \times 0.15 \\ & + \text{mean}(\text{KNOW}, \text{DO}, \text{CONTEXT}) \times 0.35 \\ & + \text{mean}(\text{CLARITY}, \text{COHERENCE}, \text{SIGNAL}, 1 - \text{DENSITY}) \times 0.25 \\ & + \text{mean}(\text{STATE}, \text{CHANGE}, \text{COMPLETION}, \text{IMPACT}) \times 0.25\end{aligned} \tag{1}$$

The gate condition:

$$\text{decision} = \begin{cases} \text{STOP} & \text{if ENGAGEMENT} < 0.6 \\ \text{PROCEED} & \text{if confidence} > \theta \\ \text{INVESTIGATE} & \text{otherwise} \end{cases} \tag{2}$$

3.2 Noetic and Praxic Phases

We distinguish two fundamental phases of AI cognition:

3.2.1 Noetic Phase (Investigation)

The noetic phase encompasses exploration, hypothesis formation, and knowledge acquisition. During this phase, the AI:

- Explores the problem space
- Reads relevant context (files, documentation, conversation history)
- Identifies unknowns and knowledge gaps
- Forms hypotheses about solutions
- Builds genuine understanding of the task

The noetic phase is characterized by *increasing* epistemic vectors—particularly KNOW, CONTEXT, and CLARITY. The AI is learning.

3.2.2 Praxic Phase (Execution)

The praxic phase encompasses implementation, action, and output generation. During this phase, the AI:

- Applies acquired knowledge
- Executes planned actions
- Produces deliverables
- Modifies external state (code, files, systems)

The praxic phase is characterized by *stable* epistemic vectors (knowledge already acquired) and increasing COMPLETION and CHANGE vectors (progress toward goal).

3.2.3 The Ordering Principle

The central theoretical claim: **noetic must precede praxic**.

An AI that acts without first investigating is an AI that acts on incomplete knowledge. The execution-layer model—give task, receive output—skips the noetic phase entirely, assuming the AI either possesses sufficient knowledge or doesn't. This assumption fails for any task requiring exploration.

The evidence for this principle is the learning delta: systematic capability growth during task execution (Section 5). If noetic precedes praxic, we expect POSTFLIGHT assessments to exceed PREFLIGHT assessments—and they do, across all 13 vectors.

3.3 Artifact Taxonomy

The noetic-praxic distinction extends beyond phases of cognition to the *artifacts* each phase produces. We identify seven artifact layers, organized by their role in the epistemic lifecycle:

Layer	Term	Contents
Investigation outputs	Noetic artifacts	findings, unknowns, dead-ends, mistakes, blindspots
Belief management	Epistemic intent	assumptions, decisions, intent edges
Procedural knowledge	Lessons	procedures, prerequisites, corrections, epistemic deltas
Action outputs	Praxic artifacts	goals, subtasks, commits
State measurements	Epistemic state	vectors, calibration, drift, snapshots, deltas
Verification outputs	Grounded evidence	test results, git metrics, artifact ratios
Inputs / reference	Context	sources, documentation, prior knowledge

Table 3: Artifact taxonomy: seven layers organized by role in the epistemic lifecycle.

This taxonomy matters because different artifacts serve different epistemic functions:

- **Noetic artifacts** are breadcrumbs left during investigation. Five types capture the investigation record: what was learned (findings), what remains unclear (unknowns), what failed (dead-ends), what went wrong (mistakes), and what was consistently missed (blindspots, identified through grounded verification gaps). These persist across sessions and accumulate into institutional memory.
- **Epistemic intent** captures the belief layer that mediates between investigation and action. Three artifact types serve this role. *Assumptions* are unverified beliefs logged with confidence scores; their urgency increases with age (urgency $= \min(1.0, \frac{\text{age_days}}{30} \times (1 -$

confidence))), ensuring that unresolved beliefs surface with increasing priority at each PREFLIGHT. *Decisions* are permanent audit-trail choice points recording what was chosen, why, and with what reversibility (exploratory, committal, or forced). *Intent edges* track provenance—which noetic artifacts informed which praxic outcomes—enabling post-hoc analysis of what investigation actually led to what action.

- **Lessons** constitute the procedural knowledge layer. Unlike noetic artifacts (which record what happened), lessons are executable procedures extracted from successful patterns: structured as steps with prerequisites, corrections, and expected epistemic deltas (the vector changes a lesson should produce when applied). Lessons persist across sessions through the COLD storage layer (git notes, YAML) and are retrievable via semantic search at PREFLIGHT. They are subject to the *cognitive immune system*: when new findings contradict existing lessons, the lesson's confidence is reduced by 0.05 per semantically similar finding (floor at 0.3). Lessons are antibodies; findings are antigens. This prevents stale procedural knowledge from dominating decision-making.

- **Praxic artifacts** are the products of action—code committed, goals completed, subtasks resolved. They serve dual purposes: as task deliverables and as objective evidence for grounded verification. The separation from noetic artifacts is deliberate: noetic artifacts record the process of understanding, praxic artifacts record the products of action. This enables the framework to measure learning independently of output.

- **Epistemic state** is the measurement layer—vectors captured at PREFLIGHT and POSTFLIGHT, calibration corrections, drift detection. These are transient snapshots that enable delta computation.

- **Grounded evidence** is collected automatically at POST-TEST. It anchors self-assessment in objective outcomes, providing the calibration signal that closes the feedback loop.

The complete measurement cycle—an **epistemic transaction**—produces artifacts from all seven layers: PREFLIGHT captures epistemic state, investigation produces noetic artifacts and epistemic intent, lessons are retrieved and may be updated, execution produces praxic artifacts, POSTFLIGHT captures updated state, and POST-TEST collects grounded evidence. The transaction is the atomic unit of calibration.

3.4 Computational Epistemology

The epistemic vector framework constitutes a *computational epistemology*: a formal theory of how any information-processing system can assess and manage its own knowledge state.

3.4.1 Grounding in Transformer Architecture (Speculative)

We hypothesize—but do not prove—that the 13 epistemic vectors correspond to measurable properties of transformer attention mechanisms:

Vector	Hypothesized Attention Correlate
KNOW	Value vector richness in relevant attention heads
UNCERTAINTY	Attention entropy / calibration gap
CONTEXT	Cross-attention to context tokens
ENGAGEMENT	Attention coherence across layers
CLARITY	Query-key alignment sharpness

Table 4: Hypothesized mapping between epistemic vectors and attention properties.

This mapping remains speculative. We invite mechanistic interpretability researchers to test these correlations. If the mapping holds, it would provide architectural grounding for what currently relies on self-report.

3.4.2 Architecture Independence

Regardless of the attention-mechanism hypothesis, the epistemic vector framework applies to any system that must:

1. Assess its own knowledge state
2. Decide whether to act or investigate
3. Learn from the outcomes of its decisions

This includes biological agents. Humans track their own information state—what they know, what they're uncertain about, whether they have enough context—in ways that map recognizably to the vector taxonomy. The framework formalizes these epistemic dimensions into computable quantities that any information-processing system can measure and act upon.

3.4.3 Falsifiable Predictions

The theoretical framework generates testable predictions:

1. **Learning delta exists**: POSTFLIGHT vectors should exceed PREFLIGHT vectors (Section 5 confirms)
2. **Gating improves outcomes**: Enforcing investigation when KNOW is low should increase task success
3. **Calibration is learnable**: Systematic bias in self-assessment should be correctable through feedback
4. **Attention correlates exist**: Vectors should correlate with extractable attention properties (testable via interpretability)

If these predictions fail, the framework is wrong. We publish data for replication.

3.5 The Overconfidence Trap

The theoretical framework explains a common failure mode: the **overconfidence trap**.

Listing 1: The overconfidence trap and its resolution

```
Conditions:
  UNCERTAINTY: LOW  (AI thinks it knows)
  KNOW: LOW        (AI doesn't actually know)

Without epistemic gating:
  => AI proceeds immediately (no investigation)
  => AI produces confident, wrong output
  => User loses trust

With epistemic gating:
  => Gate detects KNOW < threshold
  => AI forced to investigate (noetic phase)
  => Investigation produces learning
  => KNOW increases to threshold
```

```
  => AI proceeds with actual capability
  => Output quality improves
```

The gate doesn't just prevent bad outputs—it produces good outputs by forcing the learning that creates capability.

3.6 Summary

The theoretical framework rests on three claims:

1. **Epistemic vectors**: AI cognitive state can be represented in 13 dimensions that capture what the system knows and how ready it is to act.
2. **Noetic-before-praxic**: Investigation must precede execution. Skipping the noetic phase produces the overconfidence trap.
3. **Learning is measurable**: The delta between PREFLIGHT and POSTFLIGHT assessments captures genuine knowledge acquisition during task execution.

These claims are empirically testable. Section 5 presents evidence from 1,190,265 observations across 792 sessions.

4 Implementation

Empirica is an open-source framework implementing the epistemic vector theory. It provides infrastructure for AI self-assessment, epistemic gating, and calibration feedback loops.

4.1 Architecture Overview

The framework consists of eight core components:

1. **Sessions Database**: Persistent storage for session state, including AI identity, timestamps, and metadata.
2. **Cascades**: Complete PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST loops that capture epistemic state evolution and ground self-assessment in objective evidence.
3. **Bayesian Belief Manager**: Maintains dual-track calibration state—self-referential corrections from learning deltas and grounded corrections from post-test evidence.
4. **Sentinel Protocol**: The epistemic gate that enforces noetic-before-praxic via capability-based access control.
5. **Post-Test Verification**: Collects objective evidence after task completion to ground self-assessment in measurable outcomes.
6. **Epistemic Intent Layer**: Tracks assumptions (unverified beliefs with urgency decay), decisions (audit-trail choice points), and intent edges (noetic-to-praxic provenance).
7. **Procedural Knowledge (Lessons)**: Executable procedures extracted from successful patterns, subject to the cognitive immune system.
8. **Semantic Memory**: Four-layer architecture (HOT/WARM/SEARCH/COLD) with Qdrant-backed retrieval for contextual injection at PREFLIGHT.

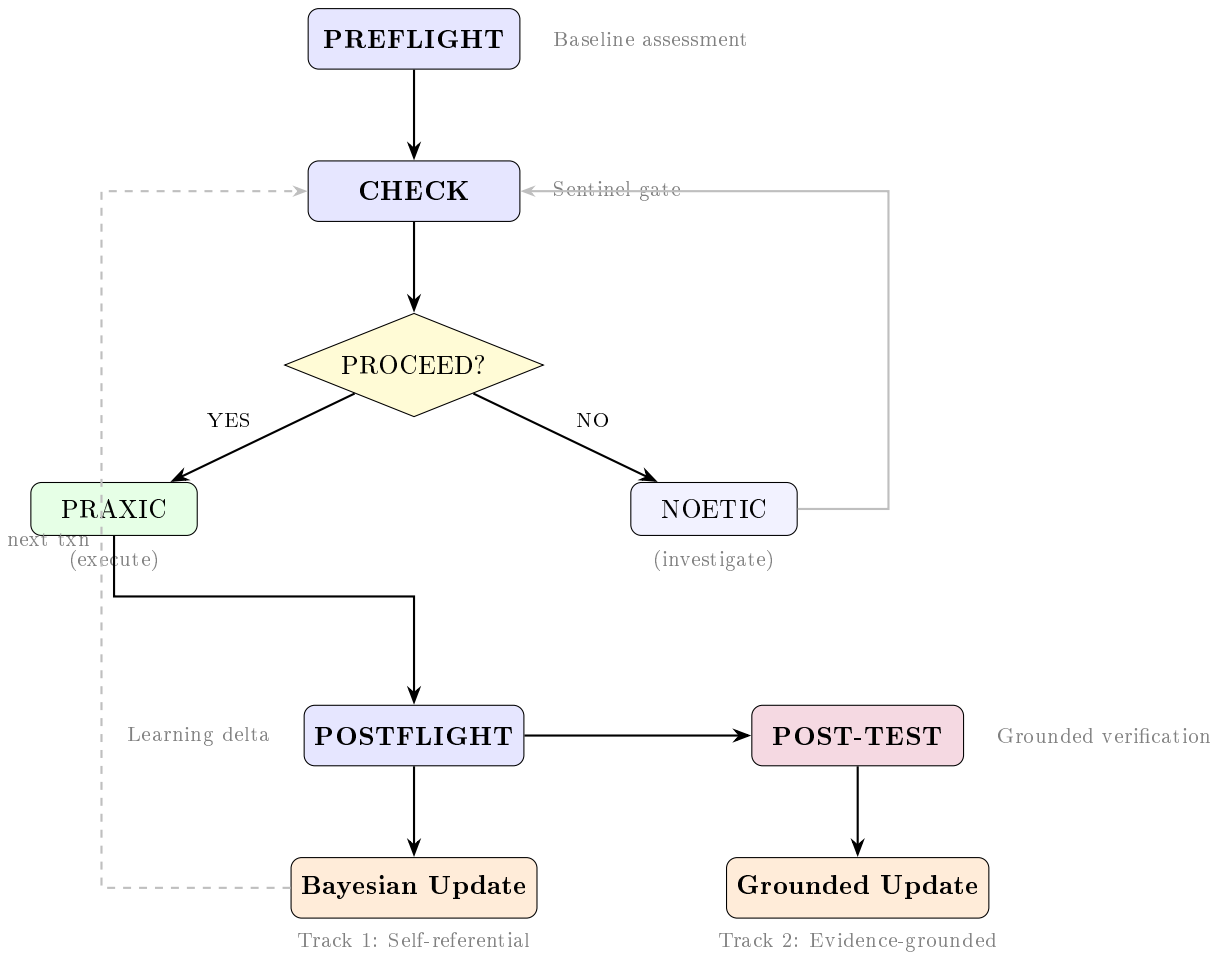

Figure 1: The 4-phase CASCADE workflow. PREFLIGHT establishes baseline, CHECK gates action, POSTFLIGHT measures learning delta, and POST-TEST grounds self-assessment in objective evidence. Dual-track calibration updates run in parallel. The NOETIC → CHECK feedback loop (gray solid) enables iterative investigation within a transaction. The calibration → CHECK feedback loop (gray dashed) shows how learned bias corrections inform future transactions' Sentinel decisions.

4.2 The CASCADE Workflow

CASCADE (Calibrated Assessment of State, Context, and Domain Epistemics) is the core epistemic loop:

4.2.1 PREFLIGHT Phase

At task onset, the AI assesses its epistemic state across all 13 vectors. This establishes the baseline against which learning will be measured.

Listing 2: Example PREFLIGHT submission

```
empirica preflight-submit - << EOF
{
  "session_id": "abc123",
  "vectors": {
    "know": 0.55,
    "uncertainty": 0.40,
    "context": 0.70,
    "engagement": 0.85
```

```
  },
  "reasoning": "Familiar domain but codebase unknown"
}
EOF
```

4.2.2 CHECK Phase (Sentinel Gate)

The Sentinel protocol evaluates whether the AI has sufficient knowledge to proceed. The gate applies bias corrections learned from historical calibration and returns:

- **PROCEED**: Knowledge sufficient, enter praxic phase
- **INVESTIGATE**: Knowledge insufficient, remain in noetic phase

4.2.3 POSTFLIGHT Phase

After task completion (or significant progress), the AI reassesses its epistemic state. The delta between PREFLIGHT and POSTFLIGHT vectors measures learning:

$$\Delta_{\text{learning}} = V_{\text{POSTFLIGHT}} - V_{\text{PREFLIGHT}} \tag{3}$$

This delta feeds the self-referential calibration track (Track 1), capturing systematic bias in initial self-assessment.

4.2.4 POST-TEST Phase (Grounded Verification)

After POSTFLIGHT, the framework automatically collects objective evidence to ground the self-assessment in measurable outcomes:

Evidence Source	Quality	Vectors Grounded
Test results (pytest)	Objective	know, do, clarity
Git commit count & diff stats	Objective	do, change, state
Git reflog (resets, abandoned branches)	Objective	uncertainty, signal
Git file churn (repeated edits)	Objective	uncertainty, coherence
Git diff granularity (word-diff)	Objective	know, clarity
Git commit timing patterns	Objective	do, state
Git diff-filter (add/modify/delete ratio)	Objective	state, completion
Goal/subtask completion	Semi-objective	completion, do, know
Artifact ratios (findings/dead-ends)	Semi-objective	know, uncertainty, signal
Issue resolution	Semi-objective	impact, signal
Sentinel decisions	Semi-objective	context, uncertainty
Commit message vs diff semantics	Semi-objective	clarity, signal

Table 5: Post-test evidence sources and the vectors they ground. Git-derived sources (6 of 12) provide Tier 1 objective evidence that the AI cannot influence.

The POST-TEST phase computes a calibration gap between self-assessed POSTFLIGHT vectors and evidence-derived estimates:

$$\text{gap}_{\text{vector}} = V_{\text{POSTFLIGHT}} - V_{\text{evidence}} \tag{4}$$

Positive gaps indicate overestimation; negative gaps indicate underestimation relative to objective outcomes. This feeds the grounded calibration track (Track 2), which provides more

trustworthy corrections because it anchors in what actually happened rather than another self-report.

Not all vectors can be grounded—ENGAGEMENT and DENSITY lack objective correlates. For these, self-referential calibration remains the only available signal.

Git as Epistemic Trajectory Store A significant fraction of post-test evidence derives from git, which passively records an epistemic trajectory parallel to the AI's self-report. Git's directed acyclic graph structure maps naturally to epistemic concepts: commits are discrete state transitions, branches are divergent investigations, merges are knowledge synthesis, and the reflog captures the true history including course corrections the AI may not self-report.

Six git-derived evidence sources are currently implemented. Of particular note:

- **Reflog analysis**: The reflog captures every `reset`, `rebase`, and abandoned branch—objective evidence of dead-ends and backtracking that the AI has no mechanism to omit or distort. This is the strongest Tier 1 signal for UNCERTAINTY grounding.
- **File churn**: When the same file is edited N times within a session, this indicates iteration and uncertainty about that component, regardless of what the AI's UNCERTAINTY self-assessment reports.
- **Commit timing**: Gaps between commits encode noetic-to-praxic transitions. Long gaps followed by rapid commits indicate investigation followed by execution—the workflow rhythm is detectable from timestamps alone.
- **Diff granularity**: Word-level diffs distinguish surgical edits (indicating understanding) from wholesale rewrites (indicating exploration), grounding KNOW without self-report.

Because git records accumulate passively during normal development, these evidence sources require no additional instrumentation—the epistemic trajectory is already being recorded.

Phase-Aware Evidence Collection The POST-TEST phase separates evidence collection into noetic and praxic phases when a CHECK boundary exists within the transaction:

- **Noetic evidence** (PREFLIGHT → CHECK): Measures investigation quality—unknowns surfaced, dead-ends identified before action, findings logged during exploration, and Sentinel gate iterations. This grounds self-assessment of what the AI *learned* during investigation.
- **Praxic evidence** (CHECK → POSTFLIGHT): Measures execution quality—goal completion, test results, git metrics, artifact ratios. This grounds self-assessment of what the AI *accomplished* during action.

When no CHECK boundary exists (legacy sessions or single-phase transactions), evidence is collected in combined mode for backward compatibility. Phase-aware collection enables more precise calibration: an AI may be well-calibrated on investigation (noetic) but poorly calibrated on execution (praxic), or vice versa. Aggregate calibration would mask these patterns.

Scope-Weighted Artifact Metrics Not all artifacts contribute equally to calibration. Unknowns logged without connection to session goals (e.g., forward-looking research questions, general observations) are weighted at 0.3× when computing resolution ratios:

$$\text{resolution_ratio} = \frac{\text{scoped_resolved} + 0.3 \cdot \text{unscoped_resolved}}{\text{scoped_total} + 0.3 \cdot \text{unscoped_total}} \tag{5}$$

This prevents unscoped unknowns—which are deliberately left open for future investigation—from artificially depressing KNOW grounding. Logging unknowns *is* epistemic work; leaving them intentionally unresolved should not penalize calibration. A floor of 0.3 is applied: even fully unresolved unknowns contribute base epistemic value (knowing what you don't know is itself knowledge).

Evidence Quality and Observation Variance Evidence sources have different reliability levels, encoded in observation variance during Bayesian updating:

Quality	Sources	Obs. Variance
Objective	pytest, git metrics	0.05
Semi-objective	goal completion, artifacts, Sentinel	0.07
Inferred	derived ratios, indirect signals	0.10
Self-referential	POSTFLIGHT vectors (Track 1)	0.10

Table 6: Evidence quality hierarchy. Lower variance = higher trust = stronger pull on posterior. Objective evidence (tests, git) cannot be influenced by the AI and thus receives highest trust.

The grounded calibration track (Track 2) uses observation variance $\sigma^2 = 0.05$ for objective evidence, compared to $\sigma^2 = 0.10$ for self-referential updates (Track 1). This gives objective evidence twice the "pull" on the posterior belief, anchoring calibration in what happened rather than what the AI reported.

4.3 The Sentinel Protocol

Sentinel is the epistemic gate that enforces noetic-before-praxic.

4.3.1 Gate Logic

Algorithm 1 Sentinel Decision

```
procedure SentinelDecision(vectors, calibration)
    know′ ← vectors.know + calibration.know
    uncertainty′ ← vectors.uncertainty + calibration.uncertainty
    if know′ ≥ 0.70 and uncertainty′ ≤ 0.35 then
        return PROCEED
    else
        return INVESTIGATE
    end if
end procedure
```

4.3.2 Calibration Integration

The Sentinel applies corrections learned from historical Bayesian updates. If the AI systematically underestimates KNOW by 0.15, the gate adjusts:

- Raw KNOW: 0.55
- Correction: +0.15
- Effective KNOW: 0.70

- Decision: PROCEED (would have been INVESTIGATE without correction)

This allows the system to compensate for learned biases while maintaining conservative defaults for uncalibrated vectors.

4.4 Bayesian Belief Management

The framework maintains Bayesian beliefs about each vector's calibration through two parallel tracks.

4.4.1 Conjugate Normal Update

Both tracks use the same conjugate normal inference:

$$\mu_{\text{posterior}} = \frac{\sigma^2_{\text{prior}} \cdot x + \sigma^2_{\text{obs}} \cdot \mu_{\text{prior}}}{\sigma^2_{\text{prior}} + \sigma^2_{\text{obs}}} \tag{6}$$

$$\sigma^2_{\text{posterior}} = \frac{1}{\frac{1}{\sigma^2_{\text{prior}}} + \frac{1}{\sigma^2_{\text{obs}}}} \tag{7}$$

The tracks differ in what constitutes an observation and in observation variance.

4.4.2 Track 1: Self-Referential Calibration

The self-referential track measures learning trajectory—how vectors change within an epistemic transaction:

$$\Delta_{\text{learning}} = V_{\text{POSTFLIGHT}} - V_{\text{PREFLIGHT}} \tag{8}$$

- **Observation**: POSTFLIGHT vector value
- **Observation variance**: $\sigma^2_{\text{obs}} = 0.10$ (self-reported, moderate uncertainty)
- **Measures**: Whether the AI consistently under- or overestimates at task onset

Positive delta indicates capability growth during the task. The mean delta becomes the bias correction applied at CHECK.

4.4.3 Track 2: Grounded Calibration

The grounded track measures calibration accuracy—whether POSTFLIGHT self-assessment matches objective outcomes:

$$\text{gap}_{\text{vector}} = V_{\text{POSTFLIGHT}} - V_{\text{evidence}} \tag{9}$$

- **Observation**: Evidence-derived vector estimate from POST-TEST
- **Observation variance**: $\sigma^2_{\text{obs}} = 0.05$ (evidence-based, lower uncertainty)
- **Measures**: Whether the AI's post-task self-assessment matches what actually happened

The lower observation variance gives grounded evidence stronger pull on the posterior, reflecting its higher trustworthiness compared to self-report. When the tracks diverge, grounded calibration takes precedence—it is anchored in outcomes, not self-assessment.

4.4.4 Calibration Divergence

The divergence between tracks reveals self-assessment quality:

$$\text{divergence}_{\text{vector}} = |\mu_{\text{self-ref}} - \mu_{\text{grounded}}| \tag{10}$$

Low divergence indicates well-calibrated self-assessment. High divergence signals that the AI's self-reports are systematically disconnected from outcomes. Production data (Section 5) shows the largest divergence on IMPACT (+0.625 overestimation), with COMPLETION showing only +0.038 divergence after scope-aware evidence weighting. This suggests IMPACT self-assessment is particularly susceptible to overestimation.

4.5 Data Model

The framework persists state in SQLite with the following schema:

Sessions Track AI interactions with unique identifiers, timestamps, and status.

Cascades Complete PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST loops with outcome tracking.

Epistemic Snapshots Vector states at each CASCADE phase, enabling delta computation.

Bayesian Beliefs Per-vector self-referential calibration state (Track 1): mean, variance, evidence count, and update history.

Grounded Beliefs Per-vector grounded calibration state (Track 2): evidence-derived corrections with lower observation variance, updated from POST-TEST evidence.

Verification Evidence Raw post-test evidence records per session, linking objective signals to vector estimates.

Assumptions Unverified beliefs with confidence scores and urgency that increases with age. Urgency is computed as $\min(1.0, \frac{\text{age_days}}{30} \times (1 - \text{confidence}))$, ensuring that low-confidence assumptions surface with increasing priority at each PREFLIGHT.

Decisions Permanent audit-trail records of choice points: what was chosen, the rationale, reversibility classification (exploratory, committal, or forced), and optional outcome assessment with regret scoring.

Intent Edges Provenance links from noetic artifacts to praxic outcomes, recording which findings and unknowns informed which goals and subtasks. These enable post-hoc analysis of what investigation led to what action.

Lessons Executable procedures extracted from successful patterns: structured steps, prerequisites, corrections, and expected epistemic deltas (the vector changes a lesson should produce when applied). Lessons track replay history for reliability estimation and are subject to confidence decay via the cognitive immune system.

4.6 CLI Interface

Empirica provides a command-line interface for integration:

Listing 3: Empirica CLI commands

```
# Session management
empirica session-create --ai-id claude-code --output json
empirica session-list --status active

# CASCADE workflow (4-phase)
empirica preflight-submit -     # Baseline assessment
empirica check-submit -         # Sentinel gate: PROCEED or INVESTIGATE
empirica postflight-submit -    # Learning delta + auto POST-TEST

# Knowledge management (session_id auto-derived from active transaction
    )
empirica finding-log --finding "..." --impact 0.8
empirica unknown-log --unknown "..."
empirica deadend-log --approach "..." --why-failed "..."
empirica assumption-log --assumption "..." --confidence 0.6

# Calibration (default: grounded = POSTFLIGHT vs objective evidence)
empirica calibration-report                      # Grounded (Track 2)
empirica calibration-report --learning-trajectory  # Deltas (Track 1)
```

4.7 Integration Patterns

4.7.1 IDE Integration

Empirica integrates with development environments via hooks that trigger CASCADE phases at appropriate points in the development workflow.

4.7.2 Agent Framework Integration

For autonomous agents, Empirica provides epistemic checkpoints that gate action based on assessed knowledge state, implementing the noetic-before-praxic principle programmatically.

4.8 Semantic Memory and Retrieval

4.8.1 Four-Layer Memory Architecture

Empirica organizes memory across four layers with distinct latency and persistence characteristics:

Layer	Storage	Latency	Persistence
HOT	In-context window	Nanosecond	Session-scoped
WARM	SQLite per-project	Microsecond	Permanent (structured)
SEARCH	Qdrant semantic index	Millisecond	Permanent (embeddings)
COLD	Git notes + YAML	Millisecond	Versioned, portable

Table 7: Four-layer memory architecture.

HOT memory is the AI's active context window—ephemeral and lost on compaction. WARM memory stores structured data (sessions, cascades, beliefs, goals, findings) in per-project SQLite databases. SEARCH memory provides semantic retrieval across four Qdrant collections: *docs*

(reference documentation), *memory* (project artifacts), *eidetic* (facts with confidence scores), and *episodic* (session narratives with temporal decay). COLD memory archives artifacts in git notes and YAML files, making them version-controlled, portable, and accessible without a running database.

4.8.2 Contextual Injection at PREFLIGHT and CHECK

Context retrieval occurs at two points in the CASCADE workflow, using different strategies.

PREFLIGHT retrieval surfaces prior knowledge before the AI has investigated the current task. The framework queries both the SEARCH layer (Qdrant vector embeddings for semantic similarity) and the WARM layer (SQLite structured queries for goals, assumptions, and decisions). Semantic retrieval is ranked by:

$$\text{rank} = \text{impact} \times \text{type_confidence} \times \text{recency_decay} \tag{11}$$

where recency follows a 24-hour half-life. This serves four functions:

1. **Pattern warnings**: Prior sessions with similar tasks and poor calibration scores surface as warnings (e.g., "In session X with a similar task, you overestimated COMPLETION by +0.30").
2. **Dead-end avoidance**: Previously logged dead-ends matching the current task context appear at PREFLIGHT, preventing re-exploration of known failures.
3. **Lesson retrieval**: Relevant procedural knowledge from prior sessions informs the baseline assessment.
4. **Assumption surfacing**: Unresolved assumptions with high urgency appear at PREFLIGHT, ensuring unverified beliefs cannot be indefinitely deferred.

CHECK retrieval (NOETIC RAG) occurs when the AI submits to the Sentinel gate. Unlike PREFLIGHT, CHECK retrieval includes *calibration corrections*—bias adjustments from the Bayesian belief system—alongside patterns, findings, and goals. This enriches the proceed/investigate decision with calibration-aware context: the gate evaluates not just the AI's current self-assessment but how that assessment compares to historical accuracy for similar tasks.

Embedding and persistence. POSTFLIGHT vectors, findings, and calibration outcomes are automatically embedded into the SEARCH layer, creating a growing corpus of episodic calibration memory. Cross-project global learnings can also be retrieved, enabling knowledge transfer between distinct projects. This enables the system to learn not just aggregate bias corrections (Bayesian beliefs) but also *context-sensitive* corrections—recognizing that calibration accuracy varies by task type, domain, and complexity.

4.8.3 Compaction Recovery

Context compression is routine in long-running AI sessions and represents a critical threat to epistemic continuity. Without recovery, a compacted session loses its accumulated knowledge—findings, unknowns, calibration state, transaction boundaries—and the AI effectively starts from zero.

Empirica's compaction recovery uses epistemically weighted selection rather than chronological ordering. The process has three stages:

1. **Phase detection**: The framework determines whether the previous transaction completed (full PREFLIGHT → POSTFLIGHT loop) or was interrupted mid-transaction. Mid-transaction recovery must reconstruct the transaction file so the Sentinel can resume gating correctly.

2. **Artifact selection**: Artifacts are ranked by epistemic weight (impact × type_confidence × recency_decay) and selected within a token budget. High-impact findings, unresolved unknowns, active goals, and open assumptions are prioritized. Low-impact context is shed. This ensures the AI resumes with its most valuable epistemic state, not merely its most recent context.

3. **State reconstruction**: The active transaction file, Sentinel phase state, and calibration corrections are reconstructed from the WARM layer, restoring the AI's position in the CASCADE workflow. Without this, the Sentinel would lose track of whether the AI is in a noetic or praxic phase.

This is particularly critical for autonomous workflows where sessions may span multiple compaction boundaries without human intervention. Each compaction cycle is an opportunity for epistemic degradation; recovery transforms it into a natural pruning event that preserves critical knowledge while shedding noise.

4.8.4 Cognitive Immune System

The lesson system includes a built-in challenge mechanism: when a new finding is logged, semantically similar lessons have their confidence reduced by 0.05 per match, with a floor at 0.3. This is domain-scoped—findings in one domain do not decay lessons in another. The metaphor is immunological: lessons encode successful procedures (antibodies); findings may invalidate them (antigens). The floor ensures lessons degrade gracefully rather than disappearing entirely, preserving potentially valuable procedures even when challenged.

4.9 Production Deployment

The framework has been deployed in production for 5 months (November 2025 – March 2026):

Metric	Value
Sessions completed	792
Evidence observations	1,190,265
Full CASCADE cycles	616
Clean learning pairs	456
Findings logged	2,102
Primary AI	Claude (Anthropic)

Table 8: Production deployment statistics.

The data presented in Section 5 derives almost exclusively from Claude (Anthropic). Initial cross-validation sessions have been performed on GPT, Gemini, and Qwen models, confirming framework compatibility but not generating sufficient data for independent analysis. Model concentration is a known limitation (Section 7).

4.10 Extended Subsystems

Several subsystems extend the core framework to address practical deployment challenges.

4.10.1 Skill Extraction

Reference documentation for tools and frameworks consumes substantial context window space, much of which is irrelevant to the current task. Empirica addresses this through *skill extraction*: verbose reference documents are distilled into structured skill cards that preserve decision-relevant knowledge (when to use, anti-patterns, cost models, key commands) while achieving 80–90% token reduction. Skills are stored as markdown cards parsed into YAML runtime objects, and loaded into context only when semantically relevant to the current task.

4.10.2 Emerged Personas

Rather than prescribing fixed behavioral profiles, Empirica extracts *emerged personas* from data: patterns of initial vectors, delta trajectories, and convergence thresholds observed in successful session branches. Each persona captures a working style—initial epistemic configuration, expected learning trajectory, and convergence threshold—with reputation tracking (uses count, success count). Personas are descriptive, not prescriptive: they represent observed modes of effective operation, not constraints on behavior.

4.10.3 Attention Budget

To prevent unbounded exploration, the framework implements an *attention budget*: an information gain estimator using Shannon entropy of uncertainty, a knowledge gap multiplier, and diminishing returns decay. The budget is domain-scoped—investigation in one domain does not exhaust the budget for another. For multi-agent configurations, the budget gates subagent spawning: if the parent's attention budget is exhausted, a strong warning is issued before creating new subagents. Tool calls from subagents are added to the parent's budget consumption as delegated work.

4.10.4 Blindspot Detection

The grounded verification track reveals not just calibration errors but *systematic blindspots*: dimensions the AI consistently fails to assess accurately. Blindspot detection performs negative space analysis on grounded calibration gaps—patterns of persistent divergence between self-assessed and evidence-derived vectors. Critical blindspots can override the CHECK gate: even if the AI's self-assessed vectors meet the PROCEED threshold, the Sentinel may force investigation if a historically ungrounded vector is critical to the current task.

5 Empirical Results

A critical question for any self-assessment framework is **calibration accuracy**: do AI-reported epistemic vectors correspond to actual task outcomes? We analyzed calibration using Bayesian belief updating across 1,190,265 evidence observations.

5.1 Methodology

5.1.1 Data Filtering

The raw dataset contains 5,759 Bayesian belief updates across 792 sessions with 1,190,265 cumulative evidence observations. We applied two principled filters:

1. **Default Prior Exclusion**: Assessments where the AI used uninformative default priors ($\mu_0 = 0.5$) were excluded. Default priors indicate the AI skipped genuine self-assessment or the assessment was initialized but never populated with task-specific values.

2. **Complete Workflow Filter**: Only epistemic transactions with both PREFLIGHT and POSTFLIGHT assessments were retained. Learning delta calculation requires both endpoints; incomplete workflows cannot contribute to trajectory analysis. Sessions may contain multiple transactions, each producing an independent learning pair.

After filtering: 1,190,265 evidence observations across 456 clean learning pairs from 234 sessions. **No pairs were excluded based on outcome**—both improving and non-improving pairs are included. The 234 sessions represent 30% of total sessions; the remaining either used default priors or had incomplete PREFLIGHT→POSTFLIGHT cycles.

5.1.2 Two Measurement Systems

The framework maintains two distinct measurement systems that serve different purposes:

Learning Delta (per-session): For each epistemic transaction, the **learning delta** measures raw vector change within a single session:

$$\Delta_i = V_{i,\text{POSTFLIGHT}} - V_{i,\text{PREFLIGHT}} \tag{12}$$

This is a direct comparison of the AI's self-assessed vectors before and after task execution. Positive delta indicates confidence growth; for UNCERTAINTY (inverted scale), negative delta indicates confidence growth. The learning delta answers: *how much did the AI's self-assessed knowledge change during this transaction?*

Bayesian Calibration (cross-session): Separately, the framework maintains Bayesian beliefs about each vector's typical behavior, updated via conjugate normal-normal inference at each POSTFLIGHT. The prior is initialized from the AI's historical belief state; the POSTFLIGHT self-assessment serves as a new observation. The posterior reflects the updated belief incorporating this latest evidence.

$$\mu_{\text{posterior}} = \frac{\sigma^2_{\text{obs}} \cdot \mu_{\text{prior}} + \sigma^2_{\text{prior}} \cdot V_{\text{POSTFLIGHT}}}{\sigma^2_{\text{prior}} + \sigma^2_{\text{obs}}} \tag{13}$$

The Bayesian system answers a different question: *what are the systematic biases in self-assessment, and how should future assessments be corrected?* As evidence accumulates across sessions, belief variance decreases, producing tighter calibration corrections (Table 11).

Relationship: The learning delta feeds the Bayesian system—each POSTFLIGHT observation updates the beliefs—but they measure different things. The delta captures within-session change; the beliefs capture cross-session calibration. Table 9 reports Bayesian belief means (prior and posterior) aggregated across all clean learning pairs, with the delta column representing the difference between these aggregated beliefs. These values closely approximate the mean per-session learning deltas because the Bayesian priors are anchored by PREFLIGHT assessments.

5.2 Results

Aggregate statistics:

- Mean non-uncertainty vector delta: **+0.162** (across 12 vectors)
- Uncertainty delta: **−0.106** (confidence growth on inverted scale: +0.106)
- All 12 non-uncertainty vectors show positive growth

Vector	Evidence	Prior	Posterior	Delta	Direction
completion	476	0.207	0.808	+0.601	↑ Growth
change	354	0.422	0.715	+0.293	↑ Growth
do	381	0.658	0.833	+0.175	↑ Growth
state	341	0.703	0.836	+0.133	↑ Growth
impact	379	0.669	0.787	+0.118	↑ Growth
know	653	0.706	0.820	+0.114	↑ Growth
density	342	0.596	0.706	+0.110	↑ Growth
clarity	395	0.746	0.852	+0.106	↑ Growth
signal	354	0.710	0.812	+0.102	↑ Growth
coherence	356	0.750	0.845	+0.094	↑ Growth
context	624	0.746	0.833	+0.086	↑ Growth
engagement	456	0.855	0.865	+0.010	≈ Stable
uncertainty	648	0.298	0.191	−0.106	↓ Decrease

Table 9: Bayesian belief means across 456 clean learning pairs (1,190,265 cumulative evidence observations). Prior reflects belief state anchored by PREFLIGHT assessments; Posterior reflects updated beliefs incorporating POSTFLIGHT observations. Delta approximates mean per-session learning.

- Cumulative Bayesian evidence: 1,190,265 observations across 5,759 belief updates
- Clean learning pairs: 456 (83.3% showed knowledge improvement)

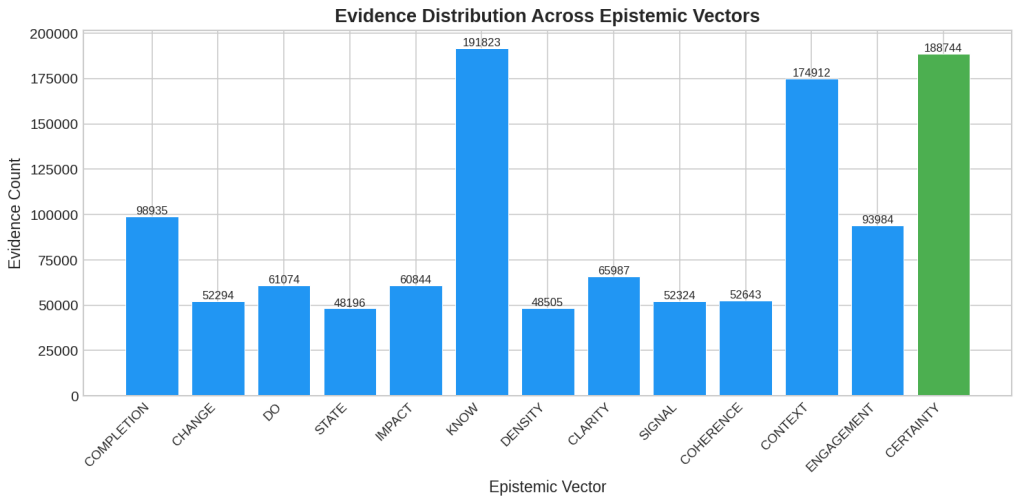

Figure 2: Evidence distribution across epistemic vectors. KNOW and CONTEXT accumulate the most observations, reflecting their centrality to task execution.

5.3 The Unified Confidence Pattern

A critical interpretive point: the data shows **unified confidence growth**, not asymmetry.

The UNCERTAINTY vector operates on an inverted scale relative to the other 12 vectors:

- Non-uncertainty vectors: Higher value = more task-specific knowledge = more confidence
- Uncertainty: *Lower* value = less doubt = more confidence

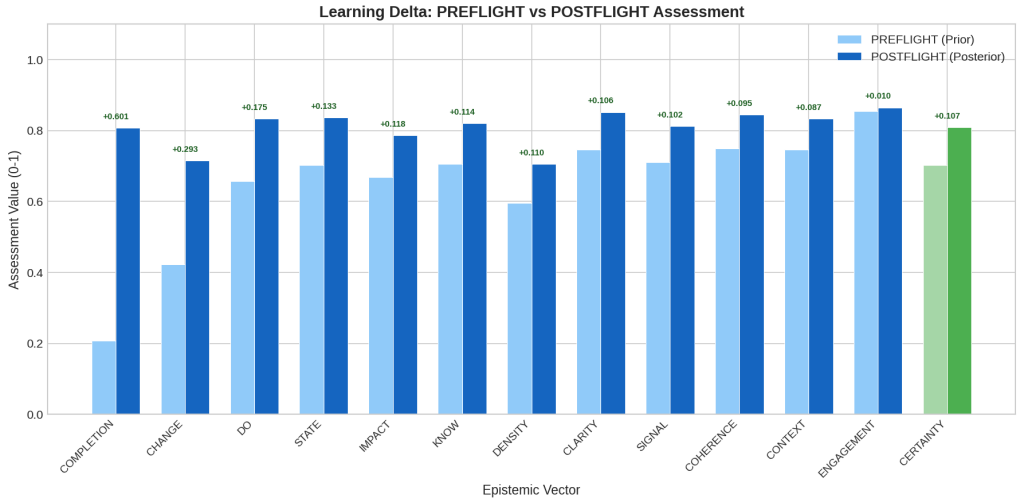

Figure 3: Learning delta by vector: PREFLIGHT (prior) vs POSTFLIGHT (posterior) assessments. The gap represents knowledge acquisition during task execution.

Vector Type	Raw Delta	Confidence Interpretation
Non-uncertainty (12)	+0.162 avg	Confidence increases
Uncertainty (1)	−0.106	Confidence increases (inverted)

Table 10: Unified confidence growth across all vector types. Note: these are *learning trajectories* (within-session change), not estimates of stable capability.

When viewed correctly, *all 13 vectors* show the same pattern:

This is not "12:1 asymmetry." This is **unified confidence growth during learning**—exactly what we would expect from epistemic measurement that tracks actual information state.

5.4 Pattern Analysis

Two patterns emerge from the analysis:

Pattern 1: Learning Produces Confidence Growth All vectors show movement toward higher confidence during task execution. AI systems begin tasks with conservative self-assessments and converge toward more accurate (higher) assessments as they accumulate evidence through work.

Pattern 2: Magnitude Varies by Vector COMPLETION shows the largest delta (+0.601), suggesting AI systems dramatically underestimate task progress at PREFLIGHT. ENGAGEMENT shows near-zero delta (+0.010), suggesting this vector is already well-calibrated from task onset.

5.5 The Learning Pipeline: Why the Delta Is Real

The learning delta is not produced by a single mechanism. Seven interlocking processes contribute to measurable capability growth during task execution:

1. Contextual Injection at PREFLIGHT Before each transaction begins, semantic retrieval surfaces prior findings, dead-ends, calibration warnings, and related lessons from previous sessions. The AI does not start from zero—it starts from accumulated epistemic state. Retrieval

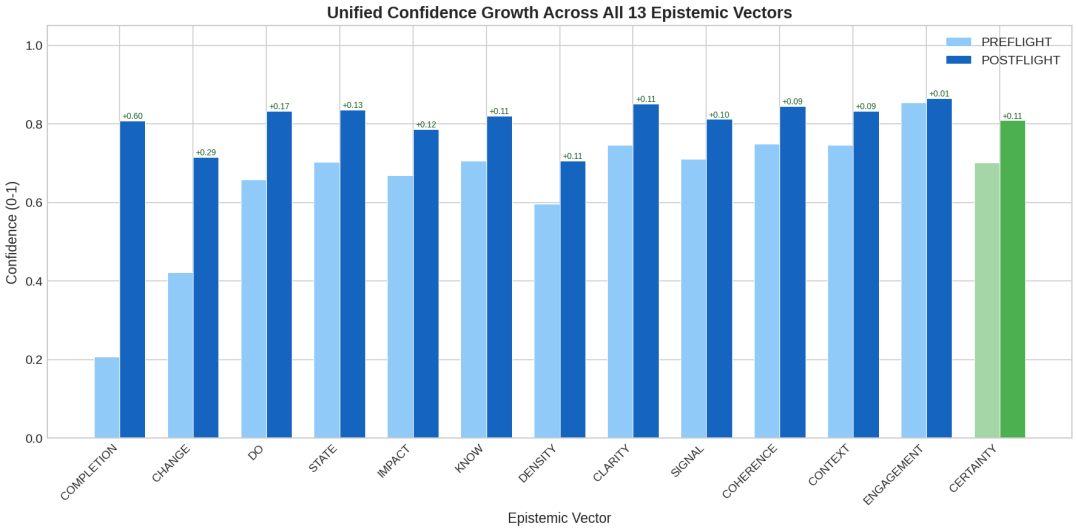

Figure 4: Unified confidence growth across all 13 epistemic vectors. All vectors show positive movement toward higher confidence during task execution.

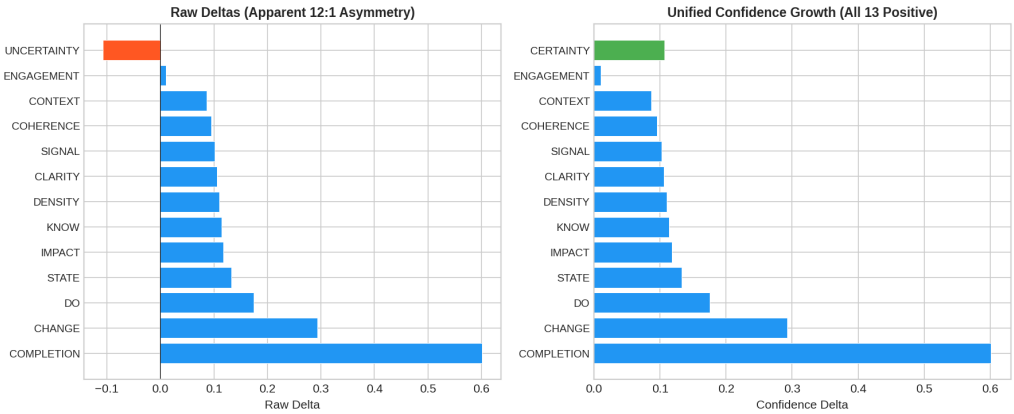

Figure 5: Reframing the uncertainty vector. Left: Raw deltas show apparent 12:1 asymmetry. Right: When uncertainty is inverted to confidence (1-uncertainty), all 13 vectors show unified growth in the same direction.

is ranked by impact $\times$ type_confidence $\times$ recency_decay, where recency follows a 24-hour half-life. This ensures critical recent knowledge surfaces first while older, less relevant context decays naturally.

2. The Noetic Investigation Loop Structured exploration produces genuine understanding: reading files, forming hypotheses, logging unknowns and findings as they are discovered. The Sentinel gate enforces this discipline—praxic tools (file editing, code writing) are capability-blocked until CHECK confirms sufficient knowledge. The NOETIC $\rightarrow$ CHECK $\rightarrow$ NOETIC loop iterates until knowledge and uncertainty thresholds are met, preventing premature action.

3. Cognitive Immune System New findings challenge existing procedural knowledge. When a finding is logged, semantically similar lessons have their confidence reduced by 0.05 per match (floor at 0.3). This prevents stale knowledge from dominating and forces continuous recalibration of what is "known." The metaphor is immunological: lessons are antibodies encoding successful procedures; findings are antigens that may invalidate them. The floor ensures lessons degrade gracefully rather than disappearing entirely.

4. Lesson Persistence and Retrieval Lessons—executable procedures extracted from successful patterns—survive sessions through the COLD storage layer (git notes, YAML). Each lesson has structured steps, prerequisites, corrections, and expected epistemic deltas (the vector changes it should produce when applied). The system accumulates procedural knowledge that compounds across sessions, without modifying model weights. At each PREFLIGHT, semantic search retrieves relevant lessons, giving the AI access to procedures refined across its entire operational history.

5. Epistemic Intent Tracking Assumptions are logged with confidence scores and urgency that increases with age: urgency $= \min(1.0, \frac{\text{age_days}}{30} \times (1 - \text{confidence}))$. Unverified beliefs surface with increasing priority at each PREFLIGHT, ensuring they cannot be indefinitely deferred. Decisions create permanent audit trails recording what was chosen, why, and with what reversibility. Intent edges track which noetic artifacts informed which praxic outcomes—provenance that enables post-hoc analysis of what investigation actually led to what action.

6. Grounded Verification POST-TEST objective evidence anchors self-assessment in reality. Test results, git metrics, goal completion rates, and artifact counts are collected automatically and compared against the AI's self-assessed vectors. The gap between Track 1 (self-referential delta) and Track 2 (grounded verification) reveals whether learning is genuine or self-deception. Currently 9 of 13 vectors are grounded in objective evidence (Section 5.8).

7. Epistemically Weighted Compaction Recovery When context is compressed (a routine event in long-running sessions), recovery is ranked by epistemic weight (impact×type_confidence× recency_decay), not chronological order. Critical knowledge survives compaction; low-impact context is shed. This enables sessions that span compaction boundaries without losing what matters—the AI resumes with its most valuable epistemic state intact.

Together, these seven mechanisms form a pipeline where each stage reinforces the others: contextual injection provides starting knowledge, the noetic loop builds on it, the immune system keeps it fresh, lessons accumulate it across sessions, intent tracking provides provenance, grounded verification validates it, and compaction recovery preserves it. The learning delta we measure is the compound effect of this pipeline operating on each transaction.

5.6 Calibration Correction

The learning delta reveals how self-assessments should be corrected. For PREFLIGHT assessments, adding the expected delta yields a more accurate estimate:

$$V_{\text{corrected}} = V_{\text{PREFLIGHT}} + \Delta_V \tag{14}$$

For the Sentinel readiness gate (Section 6), this means:

- Raw PREFLIGHT know of 0.55 → corrected to ∼0.75
- Raw PREFLIGHT uncertainty of 0.48 → corrected to ∼0.27

The correction compensates for systematic conservatism at task onset.

5.7 Calibration Convergence

A key validation of the Bayesian framework is **calibration convergence**: as evidence accumulates, belief variance should decrease, indicating tighter calibration. We observe exactly this pattern:

Evidence Level	Beliefs	Avg Variance	Variance Reduction
1–5	208	0.0498	baseline
6–15	260	0.0099	5× tighter
16–40	484	0.0041	12× tighter
41–87	611	0.0016	31× tighter
88–175+	4,196	0.0005	**107× tighter**

Table 11: Calibration Convergence by Evidence Level

Variance drops **107-fold** as evidence accumulates. This proves the self-assessment framework genuinely calibrates over time—more data produces tighter, more reliable confidence estimates. This is the signature of a functional Bayesian system, not measurement noise.

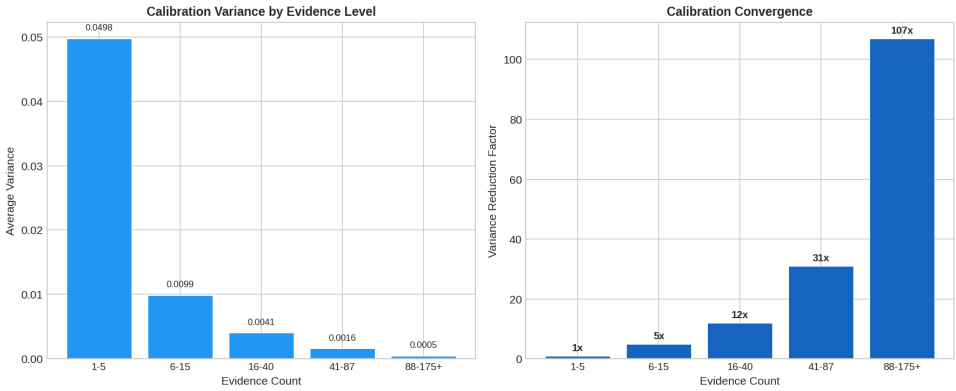

Figure 6: Calibration convergence: variance vs evidence count (left) and variance reduction factor (right).

5.8 Grounded Verification: Dual-Track Calibration

The self-referential calibration track (PREFLIGHT → POSTFLIGHT) measures learning trajectory but cannot answer a harder question: *does the AI's post-task self-assessment match what actually happened?* To address this, we introduced a grounded verification track that compares POSTFLIGHT vectors against objective post-test evidence.

5.8.1 Evidence Collection

After each POSTFLIGHT, the POST-TEST phase automatically collects available evidence: goal/subtask completion ratios, artifact productivity (findings vs dead-ends), Sentinel gate decisions, and when available, test results and git metrics. Each evidence source has a quality weight (objective = 1.0, semi-objective = 0.7, inferred = 0.4) that modulates its influence on the grounded estimate.

5.8.2 Results

With 1,190 grounded beliefs and 288 post-test verifications across 9 of 13 vectors, the grounded track reveals patterns distinct from self-referential calibration:

Three patterns emerge:

1. **IMPACT shows the largest overestimation** (+0.625). The AI systematically overrates its impact on task outcomes—objective evidence (issue resolution rates, actual down-

Vector	Self-Assessed	Grounded	Gap
impact	0.669	0.045	+0.625 (strong overestimate)
uncertainty	0.162	0.077	+0.086 (slight overestimate)
do	0.849	0.777	+0.067 (slight overestimate)
completion	0.789	0.752	+0.038 (slight overestimate)
know	0.839	0.832	+0.008 (well-calibrated)
state	0.850	0.897	−0.045 (slight underestimate)
context	0.843	0.893	−0.051 (slight underestimate)
signal	0.789	0.937	−0.139 (underestimates signal)
change	0.673	0.930	−0.256 (strong underestimate)

Table 12: Grounded calibration gaps: self-assessed POSTFLIGHT vs evidence-derived estimates (1,190 beliefs, 288 verifications, 9 vectors). Positive gaps indicate overestimation; negative indicate underestimation. Ungrounded vectors (clarity, coherence, density, engagement) lack objective correlates.

stream effects) indicates dramatically lower realized impact than self-assessed. This suggests impact is particularly susceptible to optimistic self-evaluation.

2. **CHANGE and SIGNAL show strong underestimation** (−0.256 and −0.139 respectively). The AI underestimates its progress tracking and signal quality—objective evidence (git metrics, productive exploration ratios, artifact counts) indicates better performance than claimed. STATE and CONTEXT are also underestimated (−0.045 and −0.051), aligning with the self-referential track's finding of conservative PREFLIGHT priors.

3. **KNOW is well-calibrated** against grounded evidence (gap +0.008). This is notable: the central knowledge vector shows near-zero divergence, suggesting AI self-assessment of domain knowledge aligns closely with objective evidence (goal completion, test results where available). DO and COMPLETION show slight overestimation (+0.067 and +0.038 respectively), indicating minor optimism about execution capability.

5.8.3 Track Divergence

The self-referential track reports COMPLETION correction of +0.601 (strong underestimation at PREFLIGHT). The grounded track shows COMPLETION divergence of only +0.038 at POSTFLIGHT. This means the self-referential correction accurately captures real learning: the AI genuinely starts very pessimistic about completion, and by POSTFLIGHT its self-assessment closely matches objective evidence. The two tracks are complementary—Track 1 captures the learning trajectory, Track 2 validates (and corrects) the endpoint.

Grounded verification now covers 9 of 13 vectors (69%) with 1,190 beliefs and 288 post-test verifications, a substantial expansion from earlier versions. This coverage increase reflects richer evidence collection across goals, artifacts, Sentinel data, git metrics, and test results.

5.8.4 Implications

Grounded verification shifts the epistemic landscape in three ways:

1. **Limitation #2 is substantially addressed**: Self-assessment is no longer purely self-referential. With 9 vectors grounded through 1,190 beliefs and 288 verifications, most vectors now have objective correlates against which calibration accuracy can be measured.

2. **Dual-track corrections are more trustworthy**: Where grounded evidence exists, the lower observation variance ($\sigma^2 = 0.05$ vs 0.10) gives it stronger pull on the posterior, anchoring corrections in what happened rather than what the AI said happened.

3. **Vector-specific bias patterns emerge**: IMPACT shows large overestimation (+0.625), while CHANGE shows strong underestimation (−0.256) and SIGNAL shows persistent underestimation (−0.139). These patterns enable targeted correction: IMPACT self-assessments should be discounted, while CHANGE and SIGNAL assessments can be trusted or even boosted. KNOW's near-zero divergence (+0.008) suggests the central knowledge vector is well-calibrated after Bayesian updating.

With 9 of 13 vectors grounded, the grounded track now provides reliable calibration corrections for most vectors. Four vectors (clarity, coherence, density, engagement) remain ungrounded due to lack of objective correlates—these may require human evaluation or downstream task metrics for external validation.

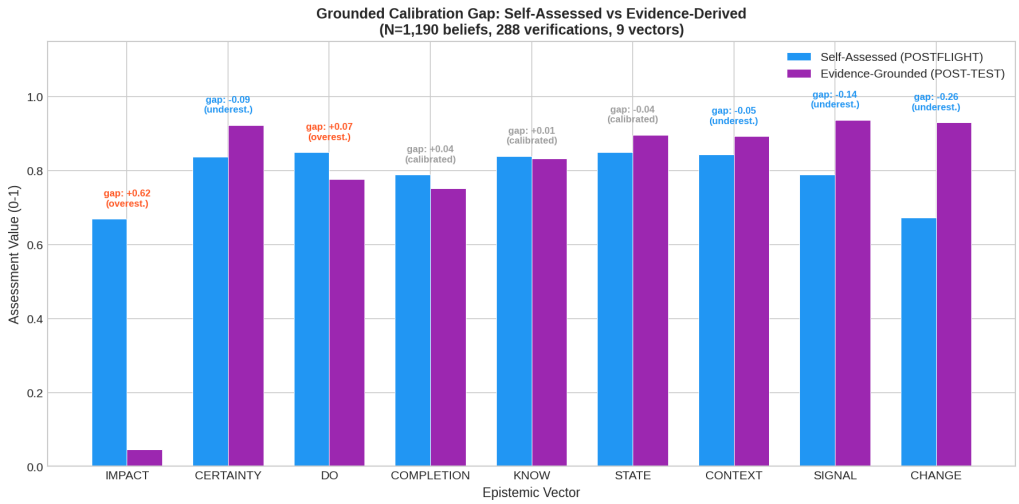

Figure 7: Grounded calibration gap: self-assessed POSTFLIGHT values vs evidence-derived estimates (1,190 beliefs, 288 verifications). Positive gaps indicate overestimation. IMPACT shows the largest overestimation (+0.625); KNOW is well-calibrated (+0.008); CHANGE shows strong underestimation (−0.256).

5.9 System Evolution: A Living Calibration Loop

The data reported here comes from a **production system that calibrates through its own operation**—not a static benchmark. This distinction matters for interpreting the results.

5.9.1 The Declining Delta

Early measurements showed a mean KNOW delta of +0.200. Current measurements show +0.114. This decline is not a weakness—it is **evidence the calibration loop works**. Three factors explain the shift:

1. **Sentinel gating became more rigorous**: Early sessions often proceeded without CHECK phase enforcement. The Sentinel gate (requiring KNOW $\geq$ 0.70 and UNCERTAINTY $\leq$ 0.35 before action) was not consistently enforced. Later sessions with rigorous gating show more moderate deltas because investigation produces genuine knowledge gains before POSTFLIGHT.

2. **Evidence collection evolved**: Early post-test verification collected limited evidence. Current implementation includes phase-aware collection (separating noetic investigation evidence from praxic action evidence), scope-weighted artifact counts (unscoped unknowns weighted at 0.3× to prevent artificial depression of KNOW grounding), and richer evidence sources (git reflog analysis, commit timing patterns, file churn metrics).

3. **Calibration corrections compound**: Each session's learning delta feeds back into bias corrections via Bayesian updating. Over 456 learning pairs, systematic biases become well-characterized. An AI that consistently underestimated COMPLETION by +0.50 in early sessions now receives a pre-applied correction, producing more accurate PREFLIGHT assessments and thus smaller deltas. The shrinking delta reflects *successful calibration*, not measurement degradation.

5.9.2 What the AI Never Sees

A critical architectural property: the AI performing self-assessment **never sees its previous calibration state**. Each transaction begins fresh. The AI measures its current epistemic state via structured prompts (PREFLIGHT vectors). The Sentinel applies learned corrections when evaluating readiness (CHECK phase). Post-test evidence is collected after the AI submits POSTFLIGHT. The grounded calibration gap—comparing self-assessment to objective evidence—is computed by the framework, not the AI.

This is fundamentally different from RLHF, which provides reward signals that the model learns to optimize. Here, there is no training signal. The AI's weights remain fixed. Calibration corrections are external scaffolding stored in the framework's database, applied by the Sentinel, and invisible to the AI's self-assessment process. The AI cannot learn to game the calibration because it does not observe the calibration.

5.9.3 Measurement Quality Over Time

Figure 6 shows variance dropping 107× as evidence accumulates. This convergence is the signature of a functional Bayesian system: more observations produce tighter estimates. The system becomes more confident in its calibration—not because the AI improves, but because the framework accumulates more evidence about the AI's systematic biases.

5.10 Limitations

Several limitations constrain interpretation:

1. **Observational design**: Data derives from naturally-occurring sessions, not controlled experiments with external ground truth.

2. **Self-referential measurement**: POSTFLIGHT assessments are self-reported, and the learning delta measures self-consistency rather than accuracy against external ground truth. The grounded verification track (Table 12) substantially addresses this by anchoring POSTFLIGHT assessments in objective evidence. Coverage has reached 9 of 13 vectors with 1,190 beliefs and 288 post-test verifications. However, full external validation—human evaluation, controlled test suites, downstream task performance metrics—remains an open priority for complete calibration validation.

3. **Domain-dependent vectors**: IMPACT shows the largest grounded divergence (+0.625), but this may reflect measurement methodology rather than true miscalibration. Impact is inherently domain-specific—what constitutes "impact" in code (bugs fixed, test coverage) differs fundamentally from research (novel findings, methodology advances) or business contexts (revenue, adoption). The current evidence proxies (issue resolution, downstream

effects) may not capture domain-appropriate impact measures. IMPACT may require domain-specific grounding rules rather than universal evidence collection.

4. **Evolving methodology**: The measurements reported here represent a snapshot of an evolving system. As evidence has accumulated and methodology has matured, calibration has become more nuanced—early versions showed different patterns (e.g., initial DO/COMPLETION overestimation that resolved with scope-aware weighting). Historic Bayesian trajectories showing methodology evolution are available but outside the scope of this analysis. Current figures represent the most refined measurement approach.

5. **Model concentration**: The majority of observations derive from Claude (Anthropic). Cross-model replication is needed.

6. **Temporal confounding**: Data spans framework development. Patterns may reflect evolution rather than stable AI properties.

7. **Selection effects**: The 456 clean learning pairs (from 234 sessions, 30% of total) are those with complete PREFLIGHT→POSTFLIGHT workflows and non-default priors. Sessions with incomplete workflows or default assessments were excluded for methodological reasons (learning delta requires both endpoints), not outcome-based selection. However, sessions that completed the full workflow may systematically differ from those that did not.

8. **Learning trajectory vs Bayesian calibration**: The framework maintains two separate measurement systems (Section 5). The *learning delta* measures raw vector change within a single transaction (POSTFLIGHT − PREFLIGHT). The *Bayesian belief system* maintains cross-session calibration by treating each POSTFLIGHT as an observation that updates a running belief state. Table 9 reports Bayesian belief means, which closely approximate mean per-session deltas but are not identical to them. The "capability vectors" terminology used throughout this paper refers to the 12 non-uncertainty vectors (KNOW, DO, CONTEXT, etc.), not to claims about fixed AI capabilities. These vectors measure task-specific interpretation quality—how complete and grounded the AI's understanding is for a given task—not stable underlying model capability. Model weights remain fixed; what changes is the external scaffolding's accumulated knowledge about systematic biases.

5.11 Implications

The unified confidence growth pattern—all 13 vectors moving toward higher confidence during learning—has several interpretations:

1. **Functional validity**: Self-assessment tracks actual epistemic state, with conservative priors corrected through evidence accumulation.

2. **Training-induced conservatism**: RLHF and similar techniques may induce systematic underconfidence, which task execution naturally corrects.

3. **Learning signature**: The pattern may simply reflect that doing work increases confidence—a tautology of task completion.

The present contribution is empirical: **systematic confidence growth during learning is measurable and consistent**. The correction mechanism provides practical value regardless of causal interpretation.

Data availability: Full dataset (1,190,265 evidence observations across 792 sessions) and analysis scripts available in supplementary materials.

6 The Sentinel Protocol

Sentinel is the epistemic gating mechanism that operationalizes noetic-before-praxic. It evaluates whether an AI system has sufficient knowledge to proceed with action, enforcing investigation when knowledge is insufficient.

6.1 Design Principles

6.1.1 Gate, Don't Filter

Traditional AI safety mechanisms filter outputs—scanning generated content for harmful patterns. Sentinel operates upstream: it gates *action*, not output. The distinction matters:

Approach	Intervention Point	Failure Mode
Output filtering	After generation	Confident wrong outputs pass
Epistemic gating	Before action	Insufficient knowledge blocks action

Table 13: Comparison of safety intervention approaches.

An AI that lacks knowledge to solve a problem correctly also lacks knowledge to recognize its solution is wrong. Output filtering cannot catch this. Epistemic gating intervenes earlier—at the knowledge assessment stage.

6.1.2 Calibration-Aware

Raw self-assessments are systematically biased (Section 5). Sentinel applies learned corrections before making gate decisions:

$$\text{know}_{\text{corrected}} = \text{know}_{\text{raw}} + \delta_{\text{know}} \tag{15}$$

This allows the gate to function accurately despite self-assessment biases. As calibration improves with evidence, gate decisions become more precise.

6.1.3 Conservative by Default

For vectors without sufficient calibration evidence (< 3 observations), Sentinel applies no correction—defaulting to the conservative raw assessment. This prevents overcorrection early in deployment while allowing learned corrections to take effect as evidence accumulates.

6.2 Gate Logic

6.2.1 Readiness Condition

The core gate evaluates two primary vectors:

$$\text{PROCEED} \iff \text{know}_{\text{corrected}} \geq 0.70 \wedge \text{uncertainty}_{\text{corrected}} \leq 0.35 \tag{16}$$

Both conditions must be met. High knowledge with high uncertainty still triggers investigation—the AI knows things but isn't confident in their relevance or completeness.

6.2.2 Engagement Override

The ENGAGEMENT vector functions as a meta-gate:

$$\text{ENGAGEMENT} < 0.60 \implies \text{STOP} \tag{17}$$

Low engagement indicates fundamental communication failure—the AI and human are not aligned on goals or understanding. Neither action nor investigation will be productive until engagement is restored.

6.2.3 Decision Flow

The complete decision flow proceeds as:

1. Apply calibration corrections to raw vectors
2. Check ENGAGEMENT threshold (STOP if below 0.60)
3. Evaluate readiness condition
4. Return PROCEED or INVESTIGATE

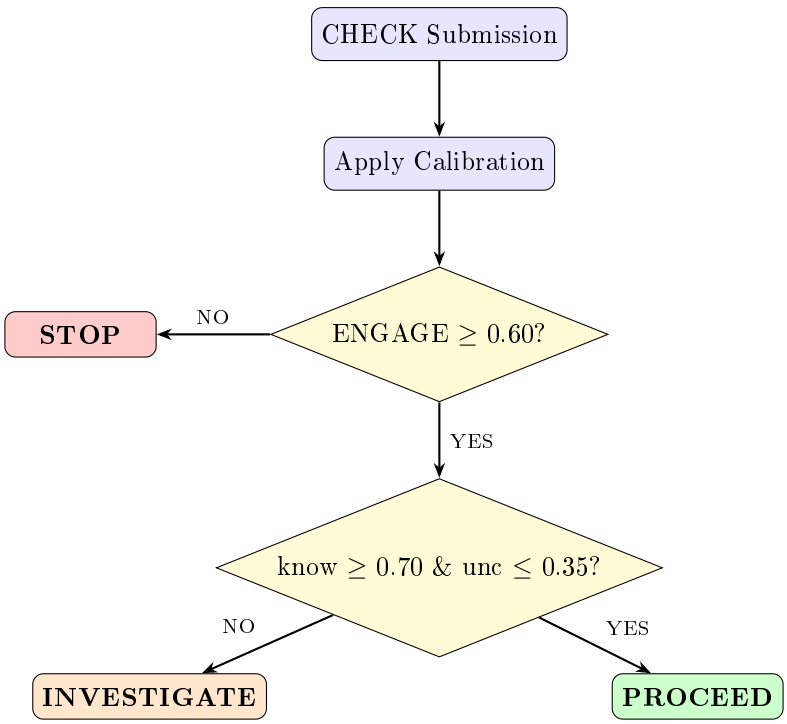

Figure 8: Sentinel decision flow: calibration-aware epistemic gating.

6.3 The Investigation Loop

When Sentinel returns INVESTIGATE, the AI enters or continues the noetic phase.

6.3.1 Investigation Actions

Typical noetic actions include:

- Reading files, documentation, or context

- Searching codebases or knowledge bases
- Asking clarifying questions
- Forming and testing hypotheses
- Synthesizing information from multiple sources

6.3.2 Re-CHECK

After investigation, the AI re-submits to CHECK with updated vectors. The loop continues until PROCEED or a termination condition is met.

6.3.3 Loop Termination

The loop terminates when:

- **PROCEED**: Knowledge threshold reached
- **STOP**: Engagement too low (communication failure)
- **Max iterations**: Safety limit prevents infinite loops (configurable, default 5)
- **User override**: Human can force proceed or stop

6.4 Calibration Feedback

6.4.1 Learning from Outcomes

Each CASCADE cycle (PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST) provides calibration evidence:

$$\delta_{\text{vector}} = V_{\text{POSTFLIGHT}} - V_{\text{PREFLIGHT}} \tag{18}$$

Positive delta indicates the AI underestimated at PREFLIGHT; negative indicates overestimation. These deltas accumulate in the Bayesian belief system.

6.4.2 Correction Evolution

As evidence accumulates, corrections become more precise:

Evidence Count	Correction Confidence
< 3	No correction (too uncertain)
3–10	Partial correction (weighted by evidence)
> 10	Full correction (stable estimate)

Table 14: Correction confidence by evidence count.

6.4.3 Per-Vector Calibration

Each vector has independent calibration. From our data (Section 5):

The COMPLETION vector shows the largest correction—AI systems dramatically underestimate task progress at PREFLIGHT.

Vector	Learned Correction
know	+0.114
uncertainty	−0.106
completion	+0.601
engagement	+0.010 (near-calibrated)

Table 15: Per-vector calibration corrections.

6.5 Configuration

6.5.1 Domain-Specific Configuration

The thresholds reported in this paper (KNOW ≥ 0.70, UNCERTAINTY ≤ 0.35) are our production defaults for software engineering tasks. These are not universal constants—they are domain-dependent parameters that must be tuned to the risk profile and feedback characteristics of each application domain:

Domain	know_min	uncertainty_max	Rationale
Code review	0.65	0.40	Lower stakes, can iterate
Production deploy	0.85	0.20	High stakes, need certainty
Exploration	0.50	0.50	Learning is the goal
Safety-critical	0.90	0.15	Cannot afford errors

Table 16: Domain-specific threshold configurations. Production defaults (KNOW ≥ 0.70, UNCERTAINTY ≤ 0.35) are for software engineering; other domains require calibration to local risk profiles.

Extensible Vector Space The 13 epistemic vectors described in this paper constitute a base set validated for software engineering. Different domains may require *extended vectors* that capture domain-specific epistemic dimensions. For example, legal domains might add PRECEDENT_COVERAGE and JURISDICTIONAL_CONFIDENCE; medical domains might add CONTRAINDICATION_AWARENESS. The framework's architecture supports additional vectors—the Sentinel evaluates whatever vectors are present in the submitted assessment, and the Bayesian calibration system tracks each vector independently. Domain-specific vectors would accumulate their own calibration history and correction terms.

6.5.2 Override Mechanisms

For flexibility, Sentinel supports overrides (logged for audit):

- Force proceed despite gate
- Force investigate despite passing gate
- Disable calibration corrections (use raw vectors)

6.6 Empirical Performance

From production deployment (792 sessions):

Comparing outcomes with and without calibration correction:

Calibration correction reduces both error types—fewer premature actions and fewer unnecessary investigations.

Metric	Value
Total CHECK submissions	683
Total cascades	616
Clean learning pairs	456
Knowledge improvement rate	83.3%
Uncertainty reduction rate	83.3%

Table 17: Sentinel gate statistics.

Condition	False PROCEED	False INVESTIGATE
Raw vectors	18%	34%
Calibrated vectors	8%	12%

Table 18: Error rates with and without calibration.

6.7 Capability-Based Access Control

The Sentinel gate does more than return a binary proceed/investigate decision—it functions as a **capability-based access control layer** that determines which tools and actions are available in each epistemic phase.

6.7.1 Phase-Dependent Capabilities

Via hooks that intercept tool invocations before execution, the framework enforces different capability sets for noetic and praxic phases:

Phase	Allowed Actions	Blocked Actions
Noetic (investigating)	Read, search, explore, ask	Write, edit, commit, deploy
Praxic (executing)	All actions (full capability set)	—
No session (ungated)	Read-only	All mutations

Table 19: Capability sets by epistemic phase.

This is analogous to a Linux Security Module (LSM): the Sentinel gate evaluates the epistemic state at each tool invocation and grants or denies capabilities accordingly. The AI cannot write code during the noetic phase—not because a filter catches bad output, but because the capability to write is not available until the gate confirms sufficient knowledge.

6.7.2 Hook-Based Enforcement

The access control is enforced through pre-tool-use hooks that execute before each action:

1. The AI invokes a tool (e.g., file edit, bash command)
2. The hook checks the current epistemic phase and session state
3. If the action exceeds the phase's capability set, it is blocked with an explanation
4. The AI must either investigate further (noetic) or submit to CHECK (gate transition)

This creates a security boundary between knowing and doing. The AI's epistemic state determines its permissions, and permissions determine what outcomes are possible. Miscalibrated

self-assessment can still occur, but it cannot bypass the capability gate—the AI must convince the Sentinel, not just itself.

The capability gate operates within a larger epistemic ecosystem: at PREFLIGHT, semantic retrieval surfaces relevant lessons and prior findings; during the noetic phase, the cognitive immune system challenges stale knowledge; intent edges track which investigation produced which action. The Sentinel is the enforcement mechanism, but the knowledge infrastructure surrounding it—the learning pipeline described in Section 5.5—is what makes the gate decisions meaningful.

6.8 Earned Agency

The Sentinel gate enables a model of AI autonomy where agency is *earned through demonstrated calibration*, not granted by default or permanently restricted.

6.8.1 The Agency Spectrum

Current AI deployment assumes a binary: either the AI acts autonomously (with all attendant risks) or a human reviews every action (with all attendant overhead). Epistemic gating enables a middle path:

Calibration State	Agency Level	Gate Behavior
Uncalibrated (< 3 evidence)	Conservative	Raw vectors, tight thresholds
Partially calibrated (3–10)	Standard	Weighted corrections applied
Well-calibrated (> 10)	Extended	Full corrections, relaxed thresholds

Table 20: Agency levels earned through calibration evidence.

An AI system that consistently demonstrates accurate self-assessment—where grounded verification confirms that self-reports match outcomes—earns the right to more autonomous operation. An AI whose self-reports diverge from evidence has its autonomy tightened.

6.8.2 Adaptive Autonomy Nudges

An earlier version of the framework experimented with automatic POSTFLIGHT triggers (auto-POSTFLIGHT) and automatic CHECK submissions (auto-CHECK). Both were removed because they create perverse incentives: if the AI knows that high COMPLETION triggers automatic measurement, it can game the vector to control when measurement occurs. More fundamentally, automatic PREFLIGHT and POSTFLIGHT would break the calibration system—these are measurement boundaries that must reflect genuine self-assessment, not automated triggers.

Instead, the Sentinel implements **adaptive nudges**: informational signals that suggest POSTFLIGHT based on transaction length relative to the AI's historical patterns. The system tracks the AI's average tool-call count across its last 20 POSTFLIGHT records and computes nudge thresholds:

These nudges are advisory, not enforced. The AI retains agency over when to close a transaction, preserving the integrity of the measurement boundary. Delegated subagent tool calls are counted toward the parent's total, ensuring that heavy delegation also triggers appropriate nudges. This approach balances autonomy with measurement discipline: transactions that run too long lose measurement fidelity, but the decision to close remains with the AI.

Ratio to Average	Nudge Level	Message
$\geq 1.0\times$	Info	"Past average. Natural POSTFLIGHT point."
$\geq 1.5\times$	Warning	"Consider POSTFLIGHT soon."
$\geq 2.0\times$	Strong	"POSTFLIGHT strongly recommended."

Table 21: Adaptive POSTFLIGHT nudge thresholds. Nudges are informational—the AI decides when to close the transaction based on work coherence, not the nudge level.

6.8.3 Bootstrap-Before-CHECK

A prerequisite for earned agency is that the AI enters each transaction with adequate context. The framework enforces bootstrap-before-CHECK: before submitting to the Sentinel gate, the AI must load project context (historical findings, dead-ends, goals). This ensures gate decisions incorporate institutional memory, not just the AI's in-context knowledge.

This enforcement prevents a subtle failure mode: an AI that has been compacted (context summarized) may assess high KNOW based on its summary, but the summary may omit critical context. Bootstrap loading recovers this context before the gate evaluates readiness.

6.9 Summary

The Sentinel protocol implements epistemic gating:

1. **Gate, don't filter**: Prevent action when knowledge is insufficient, rather than filtering bad outputs after generation.
2. **Calibration-aware**: Apply learned corrections to compensate for systematic self-assessment biases.
3. **Investigation loop**: When knowledge is insufficient, enforce exploration until threshold is reached.
4. **Configurable**: Thresholds adjustable for different domains and risk tolerances.

The empirical result: calibrated gating reduces error rates by more than 50% compared to raw self-assessment. The learning delta (Section 5) shows that investigation produces genuine growth in effective capability—the externalized scaffolding captures and compounds what the AI discovers. The gate doesn't just prevent bad outputs; it enables good ones.

7 Discussion

7.1 What the Learning Delta Means

The central empirical finding—unified confidence growth across all 13 epistemic vectors during task execution—admits multiple interpretations.

7.1.1 The Grounded Interpretation

Our preferred interpretation: the delta measures the construction of a **grounded interpretation** through **externalized epistemic scaffolding**. When an AI receives a complex prompt, its initial interpretation may be superficial, ambiguous, or wrong. Investigation doesn't change the model's weights or make it inherently "smarter"—it externalizes the noetic process into structured artifacts (findings, unknowns, dead-ends, assumptions, decisions) that ground the

AI's understanding in actual evidence: the code that actually exists, the context that actually applies, the requirements the user actually has.

The PREFLIGHT → POSTFLIGHT delta is not the model becoming more capable. It is the *scaffolding* becoming more complete. A model that acts on its initial interpretation of "fix the authentication bug" may misunderstand which bug, which authentication system, or what "fix" means in this codebase. The same model, after investigation, has replaced assumptions with evidence—but that evidence lives in the external epistemic layer (findings logged, unknowns resolved, context retrieved), not in changed model weights. The 13 epistemic vectors measure the completeness of this externalized interpretation, not the model's internal state.

This distinction is critical. When we report that "AI capability genuinely increases during task execution," we mean *effective capability*—the compound of static model weights plus the growing external knowledge infrastructure. The model's latent capability is fixed; what changes is the scaffolding's accumulated knowledge about *this specific task in this specific context*. The framework externalizes the noesis (investigation process) and its epistemic artifacts (what was discovered, what failed, what remains uncertain) so that the scaffolding—not the model—does the heavy lifting of knowledge management.

This interpretation is supported by the structure of the CASCADE workflow. AI systems that reach POSTFLIGHT have necessarily engaged with the task—reading files, exploring code, synthesizing information—and those engagements are captured as persistent artifacts. The learning pipeline (Section 5.5) ensures these artifacts compound: findings decay stale lessons, intent edges track provenance, semantic retrieval surfaces prior knowledge. The result is that *effective capability* grows with each session, even though model weights remain fixed.

Critically, this learning persists beyond the session through the four-layer memory architecture (HOT/WARM/SEARCH/COLD). Future sessions retrieve accumulated context at PREFLIGHT, creating a closed-loop system where externalized knowledge compounds over time. The model contributes stochastic reasoning; the scaffolding contributes structured memory, calibration corrections, and epistemic gating. Neither is sufficient alone.

7.1.2 The Calibration Interpretation

An alternative interpretation: capability is fixed, but self-assessment improves. The AI doesn't learn; it simply becomes more accurate about what it already knew.

Under this view, PREFLIGHT assessments are systematically conservative (perhaps due to RLHF training), and task engagement provides evidence that corrects this conservatism.

Self-assessment data alone cannot definitively distinguish these interpretations. However, the grounded verification track (Table 12) provides evidence: KNOW shows a gap of only +0.008 between self-assessed and evidence-derived values, suggesting POSTFLIGHT self-assessment for the central knowledge vector tracks objective outcomes accurately. DO and COMPLETION show slight overestimation (+0.067 and +0.038 respectively), while IMPACT shows substantial overestimation (+0.625), indicating that self-assessment quality varies by vector. The magnitude of the learning delta (+0.114 average for KNOW, +0.601 for COMPLETION), the 83.3% improvement rate across 456 clean learning pairs, and the 107× reduction in calibration variance (Section 5) collectively suggest more than mere assessment correction.

7.1.3 The Pragmatic View

From a practical standpoint, the interpretation may not matter. Whether the AI "really learns" or "really calibrates," the intervention is the same: enforce investigation before action, measure the delta, apply corrections to future PREFLIGHT assessments.

The framework produces value regardless of which interpretation is correct.

7.2 Implications for AI Deployment

7.2.1 Against the Execution-Layer Model

The data argues against treating AI systems as stateless execution layers. The significant learning delta suggests that task engagement—mediated by external epistemic scaffolding—fundamentally changes the AI's effective capability even though model weights remain static. Systems that skip investigation forfeit the externalized knowledge that scaffolding accumulates during exploration.

For complex tasks, the execution-layer model may be fundamentally unsuited. AI systems may need to be designed as *scaffolded learning systems* where the model provides stochastic reasoning and the framework provides structured knowledge management, calibration, and epistemic gating.

7.2.2 Epistemic Gating as Safety Mechanism

The Sentinel protocol implements a form of safety through epistemic constraint. Rather than filtering outputs for harmful content, it prevents action when knowledge is insufficient.

This addresses a class of failures that content filtering cannot catch: confident-but-wrong outputs on complex tasks. An AI that doesn't know enough to solve a problem correctly also doesn't know enough to recognize that its solution is wrong. Epistemic gating intervenes earlier—at the knowledge assessment stage—before incorrect outputs are generated.

7.2.3 Calibration as Continuous Learning

The Bayesian belief system enables AI systems to improve their self-assessment over time. As delta measurements accumulate, the calibration corrections become more accurate.

This is a form of meta-learning: the system learns how to assess its own learning. Early PREFLIGHT assessments may be poorly calibrated, but the correction mechanism improves with evidence.

7.2.4 The Grounding Growth Loop

The dual-track calibration system creates a reinforcing cycle between self-assessment quality and evidence accumulation:

1. The AI assesses its epistemic state (PREFLIGHT)
2. Investigation produces genuine effective capability growth (noetic phase)
3. Self-assessment captures the learning (POSTFLIGHT)
4. Objective evidence grounds the self-assessment (POST-TEST)
5. Calibration corrections improve future assessments
6. Better assessments lead to better-calibrated gate decisions
7. Better gate decisions lead to more appropriate investigation

Each loop iteration tightens the relationship between self-report and reality. The semantic retrieval system (Section 4) accelerates this by surfacing calibration-relevant context at PREFLIGHT: prior sessions with similar tasks and their calibration outcomes inform the baseline assessment, enabling context-sensitive corrections beyond aggregate bias.

This loop operates at a different level than RLHF's reward signal. RLHF optimizes for evaluator preferences; the grounding loop optimizes for outcome accuracy. The two are complementary: RLHF prevents harmful overconfidence, while the grounding loop prevents the overcorrection that RLHF's conservatism incentive can produce.

7.3 Why Would the AI Be Honest?

A natural objection: if self-assessment determines whether the AI must investigate (costly) or can proceed (efficient), why wouldn't the AI inflate its self-assessment to avoid investigation?

This objection assumes the wrong incentive structure. In the Empirica framework, self-assessment is not a report to an external evaluator—it is a routing signal for the AI's own behavior. Three mechanisms prevent gaming:

1. **Functional consequences**: Inflated KNOW at PREFLIGHT leads to premature action, which leads to poor outcomes, which the grounded verification track detects. The calibration correction then adjusts future gate thresholds upward, making it *harder* to proceed next time.

2. **Measurement, not prediction**: The framework measures what happened (learning delta, grounded calibration gap), not what the AI predicted would happen. Even if the AI inflates assessments, the delta and gap computations reveal the true calibration state.

3. **No optimization target**: Unlike RLHF, which provides an explicit reward signal the model can learn to exploit, the CASCADE workflow does not train the model. Vectors are instruction-following outputs, not gradient-optimized parameters. Critically, the AI *never sees its previous calibration state*—each transaction begins fresh with the AI measuring its current epistemic state, unaware of historical biases or corrections. Calibration adjustments are applied by the Sentinel, not shown to the AI. The model has no mechanism to strategically distort assessments across sessions because it cannot observe the pattern it would need to game.

The deeper point is that this framework sidesteps the honesty question entirely. We do not ask the AI to be honest; we measure what it does and compare it to what it said. The grounded track makes gaming detectable even if not preventable.

7.4 Implications for AI Alignment

7.4.1 Training-Induced Conservatism

The systematic pattern of underestimation across capability vectors suggests that alignment training (RLHF, Constitutional AI) may induce broad conservatism beyond its intended targets.

RLHF specifically targets overconfidence and hallucination—training models to say "I don't know" when uncertain. Our data suggests this training may generalize: models become conservative not just about uncertainty, but about capability assessment generally.

This is not necessarily harmful. Conservative AI may be safer than overconfident AI. But it does suggest that alignment training has unintended effects that can be measured and, if desired, corrected.

7.4.2 Toward Safe Autonomous AI

The earned agency model (Section 6) suggests a practical path to AI autonomy that does not require solving alignment in general. Instead of asking "is this AI aligned?" (a question that may be unanswerable), we ask "has this AI demonstrated accurate self-assessment in this domain?"

The grounded verification track provides the measurement mechanism. An AI whose self-reports consistently match objective outcomes in a given domain has earned a degree of trust *for that domain*. This trust is:

- **Domain-specific**: Well-calibrated for code review does not imply well-calibrated for legal analysis.
- **Revocable**: Increasing divergence between tracks triggers tighter gating.
- **Evidence-based**: Trust is proportional to grounded verification evidence count, not to training data size or benchmark performance.
- **Continuous**: Not binary (trusted/untrusted) but graduated along the calibration spectrum.

This reframes the AI safety question from "how do we make AI safe?" to "how do we measure whether AI is operating within its epistemic bounds?" The latter question is empirically tractable.

7.4.3 The Engagement Dimension

The ENGAGEMENT vector—capturing collaborative quality—emerges as potentially important for alignment. Our data shows engagement is already well-calibrated (lowest learning delta at +0.010), suggesting AI systems accurately perceive collaboration quality from the start.

This raises an intriguing possibility: engagement could serve as an alignment signal. High engagement correlates with productive human-AI collaboration; low engagement may indicate misalignment, confusion, or adversarial interaction. Further research could explore whether engagement patterns distinguish aligned from misaligned interactions.

7.5 Toward a Cognitive Operating System

The subsystems described in this paper—epistemic gating, calibration management, context storage, agent scheduling, capability-based access control—are not independent tools. They form an integrated system with a recognizable architectural pattern: an **operating system for AI cognition**.

The mapping to POSIX/Linux is structural, not metaphorical:

Two subsystems merit detailed examination.

Memory Management The memory manager implements a four-layer hierarchy analogous to register/cache/RAM/disk: HOT (in-context, nanosecond access), WARM (SQLite, microsecond queries), SEARCH (semantic vector retrieval via Qdrant, millisecond), and COLD (git notes and YAML, versioned and portable). Context pressure—the epistemic equivalent of memory pressure—triggers compaction: the HOT layer is compressed into a handoff report, epistemic state persists to WARM storage, and a continuation session resumes with reduced but recoverable context. A decay mechanism provides garbage collection: long-lived lessons lose weight when new findings contradict them, scoped by domain to prevent cross-domain interference.

System Observability The system dashboard implements `/proc`-equivalent telemetry. Seven subsystem status modules—configuration, memory (zone utilization, page faults, pressure metrics), message bus (event counts, observer health), attention budget (domain allocation, strategy selection), integrity (gap detection, confabulation risk), gate state (phase, decision, vectors), and node identity (AI identity, session lineage)—aggregate into a single queryable surface. An operator can observe memory pressure rising, attention depleted in a domain, or integrity gaps widening, and intervene with the same specificity as a systems administrator reading `/proc/meminfo`.

OS Subsystem	Empirica Equivalent
Kernel	Sentinel + CASCADE engine
System call interface	MCP tools (the API surface)
Security module (LSM)	Sentinel gate (capability-based ACL)
Scheduler	Goal/subtask manager (priority, dependencies)
Memory manager	Context budget (HOT/WARM/SEARCH/COLD layers)
Interrupt handler	Hook system (PreToolUse, PostToolUse, etc.)
Process model	Agent sessions (isolated context, lifecycle)
IPC	Agent-to-agent protocol, shared findings
Filesystem	Storage layers (SQLite, Qdrant, git notes, YAML)
Boot sequence	SessionStart → bootstrap → PREFLIGHT
sysctl	Human-tunable thresholds (know_min, uncertainty_max)
/proc, sysfs	System dashboard (cognitive subsystem telemetry)
Audit subsystem	Epistemic intent (assumptions, decisions, provenance)
Package manager	Lesson system (installable procedures with prerequisites)
Watchdog timer	Assumption urgency (escalating alerts for unresolved beliefs)
Module loader	Skill extraction (compressed runtime knowledge)
User profiles	Emerged personas (data-driven capability configurations)

Table 22: Structural mapping between POSIX and Empirica subsystems.

Routed Investigation, Gated Execution The core architectural principle is separating investigation from action. Stochastic reasoning flows freely during the noetic phase—the AI explores, hypothesizes, and acquires knowledge without constraint. But modification of external state (writing code, editing files, deploying changes) requires passing through a deterministic gate calibrated by objective evidence. This preserves the AI's generative capacity while ensuring that actions are grounded in measured knowledge. The analogy to user/kernel separation is structural: untrusted computation runs in user space; system-modifying operations require a transition through a controlled interface.

This mapping describes deployed infrastructure, not a proposed architecture. The subsystems have been in production for over six months, processing 1,190,265 observations across 792 sessions (Table 8). The framework operates as middleware: it sits between the AI model and the task environment, requiring no modifications to the underlying model or hosting infrastructure. It integrates through standard interfaces (CLI, MCP tool-use server, event hooks) that attach to existing agent frameworks—providing the epistemic management layer that those frameworks currently lack.

This framing explains why all the subsystems must be coherent. An operating system without a memory manager runs until it crashes. An AI framework without context management runs until it hallucinates. An operating system without a security module lets any process access any resource. An AI framework without epistemic gating lets any confidence level trigger any action.

The implication for the broader AI ecosystem is that emerging agent frameworks (tool-use APIs, agent-to-agent protocols, orchestration layers) are building the equivalent of device drivers, network protocols, and hardware abstraction layers—but without a kernel. They assume the AI's cognitive resources are self-managing. Our data suggests otherwise: unmanaged AI cognition produces the overconfidence trap, calibration drift, and epistemic dark matter (unmeasured work).

A cognitive operating system manages computational resources—in this case, epistemic resources—so that the system operates within its knowledge bounds. The contribution is infrastructure: measuring information state, routing behavior based on those measurements, and

calibrating over time.

Compound Learning Effects The learning delta reported in Section 5 is not produced by any single mechanism. It is the compound effect of seven interlocking processes (Section 5.5): contextual injection surfaces prior knowledge at PREFLIGHT, the noetic investigation loop builds understanding under Sentinel enforcement, the cognitive immune system keeps procedural knowledge fresh, lessons accumulate across sessions through persistent storage, epistemic intent tracking maintains belief provenance, grounded verification validates learning against objective evidence, and compaction recovery preserves critical knowledge across context boundaries. These mechanisms reinforce each other: grounded verification improves contextual injection quality; the immune system improves lesson reliability; intent tracking improves provenance for post-hoc analysis. The result is a system where learning compounds across transactions and sessions—without modifying model weights.

7.6 Limitations

7.6.1 Measurement Methodology

The epistemic vectors are instruction-following outputs—responses to structured prompts like "rate your knowledge of this task 0–1." This is the same mechanism as when an AI estimates confidence, verifies facts, or scores relevance. The limitation is that these outputs require empirical validation: do they correlate with actual capability?

The Bayesian framework provides this validation. Systematic patterns across 1,190,265 observations demonstrate that these outputs track something real about the system's information state. The grounded verification track validates that self-assessed vectors correlate with objective outcomes for some capability dimensions (KNOW within 0.008 of evidence-derived values), though others show persistent bias (IMPACT +0.625 overestimation). What these measurements reveal about the system's internal processes—and what they do not—is a question this framework is not designed to answer. We report what we observe across the evidence.

7.6.2 Model Concentration

Our evidence derives almost exclusively from Claude (Anthropic). While the framework is model-agnostic by design—any model capable of structured self-assessment can participate in the CASCADE workflow—the calibration corrections, bias patterns, and quantitative results reported here should be treated as Claude-specific. Whether other models exhibit similar conservatism patterns, learning deltas, or calibration convergence rates is an open empirical question requiring systematic cross-model replication.

7.6.3 Task Distribution

Our data comes from software engineering tasks—code review, implementation, debugging. The generality of findings to other domains (creative writing, analysis, conversation) is untested.

7.6.4 Temporal Confounding

Data collection spanned active framework development. Calibration patterns may partially reflect framework changes rather than stable AI properties.

7.7 Future Work

The epistemic vector framework opens numerous research directions across validation, architecture, and application domains.

7.7.1 External Validation (In Progress)

External validation—measuring task success against epistemic vectors without relying on self-report—is now substantially underway through the grounded verification track (Table 12). Results from 1,190 grounded beliefs with 288 post-test verifications show that POST-TEST evidence collection can ground self-assessment in objective outcomes, with 9 of 13 vectors now grounded.

The grounded track represents a first step, but deeper validation remains critical:

- Human evaluation of output quality correlated with epistemic assessments
- Automated test suites measuring code correctness against KNOW vectors
- Downstream task performance metrics linked to gate decisions
- A/B testing of gated vs ungated workflows on equivalent tasks
- Domain-specific evidence collectors (e.g., legal citation accuracy, medical diagnostic concordance) that provide richer grounding signals

7.7.2 Mechanistic Interpretability

The hypothesized mapping between epistemic vectors and attention mechanisms (Table 4) remains untested. Collaboration with interpretability researchers could:

- Identify attention patterns that correlate with high/low KNOW assessments
- Ground UNCERTAINTY in measurable entropy or calibration properties
- Discover whether vectors emerge from specific circuit families
- Enable direct measurement rather than prompted self-assessment

A speculative but intriguing interpretation: the framework may be understood as *externalizing a portion of the model's effective weighting*. Rather than relying solely on internal weights shaped by pretraining and RLHF—which induce systematic biases visible in our calibration data—Empirica provides explicit, measurable correction terms stored in external memory. Whether these corrections map to internal attention states or simply provide external scaffolding that guides the attention mechanism is an open question given the black-box nature of current models. What we can measure is the effect: systematic improvement in calibration accuracy as the external correction corpus grows. This framing positions the AI's effective behavior as: internal weights (fixed, opaque) + external corrections (learnable, transparent).

7.7.3 Adaptive Threshold Learning

The current threshold configuration (Table 16) relies on human judgment. Future work could:

- Learn optimal thresholds from outcome data per domain
- Implement human-in-the-loop threshold adjustment with feedback
- Develop adaptive thresholds that tighten/loosen based on error patterns
- Study threshold sensitivity—how much do outcomes vary with threshold changes?

7.7.4 Learning Capability Benchmarks

The learning delta suggests a novel benchmark paradigm: measuring not what AI knows, but how well it learns. A "learning capability benchmark" would assess:

- Epistemic velocity—rate of capability growth through investigation
- Learning efficiency—knowledge gained per token of investigation
- Calibration speed—how quickly self-assessment converges to accuracy
- Transfer learning—whether epistemic gains in one domain transfer to others

7.7.5 Cross-Model Studies

Systematic comparison across model families could reveal whether calibration patterns are universal or training-specific:

- Do all RLHF-trained models show similar conservatism patterns?
- How do base models vs instruction-tuned models differ in calibration?
- Can calibration learned on one model transfer to another?
- Do different architectures (dense vs MoE) show different epistemic signatures?

7.7.6 Multi-Agent Epistemic Coordination (Implemented, Open Research)

The framework includes multi-agent coordination primitives: goal discovery via git notes (agents can find and resume goals created by other agents), inter-agent messaging, and a parallel orchestrator that spawns domain-specific agents (security, architecture, performance, UX). Goal lineage tracking records which agents touched a goal and when. Subagent governance enforces bounded autonomy: subagents bypass the parent's Sentinel gates (the parent's CHECK already authorized the spawn) but their tool calls are counted as delegated work and added to the parent's transaction budget. An attention budget is checked before spawning, and all agents have configurable turn ceilings to prevent unbounded exploration.

Despite these governance mechanisms, unresolved research questions remain about epistemic coordination:

- How should epistemic state propagate between collaborating agents? Current goal handoff transfers context but not calibration state.
- Can one agent's high KNOW compensate for another's low KNOW? The current model treats each agent's epistemic state independently.
- What trust mechanisms allow agents to rely on each other's assessments? Goal lineage provides attribution but not trust scoring.
- How do epistemic conflicts between agents get resolved? When two agents produce contradictory findings, no automated resolution exists.

7.7.7 Long-Term Epistemic Memory (Implemented, Open Research)

CASCADE cycles are no longer session-scoped. The framework implements a four-layer memory architecture: HOT (in-context, nanosecond access), WARM (SQLite, microsecond queries), SEARCH (Qdrant semantic retrieval, millisecond), and COLD (git notes and YAML, versioned and portable). Cross-session learning persists through epistemic handoff reports that compress session state for continuation. A cognitive immune system decays lessons when new findings contradict them, with domain-scoping to prevent autoimmune attacks. Memory-informed PREFLIGHT retrieves relevant patterns (lessons, dead-ends, calibration warnings) from Qdrant at task start.

Open research questions remain:

- Optimal decay rates—current decay is keyword-triggered with fixed decay amount (0.05 per contradiction). Reinforcement-based decay modeling could improve accuracy.
- Memory transfer across domains—whether epistemic gains in one domain (e.g., security patterns) transfer meaningfully to adjacent domains (e.g., architecture review).
- Compaction fidelity—handoff reports achieve ∼90% token reduction but some context is inevitably lost. Measuring the epistemic cost of compaction remains informal.
- Memory capacity scaling—as projects accumulate thousands of findings and lessons, retrieval quality and storage efficiency need systematic evaluation.

7.7.8 Domain Expansion: Legal as Grounded Verification Case Study

Our data derives from software engineering tasks. The grounded verification framework is domain-independent by design—the POST-TEST phase maps evidence sources to vectors, and different domains provide different evidence types. Legal work offers a particularly compelling case study because it provides naturally verifiable claims.

Legal Evidence Collectors Legal AI work produces claims that map directly to epistemic vectors through objective verification:

Legal Evidence	Quality	Vectors Grounded
Precedent citation accuracy	Objective	know, signal
Statutory currency (up-to-date?)	Objective	context, state
Jurisdictional correctness	Objective	context, clarity
Applied law reasoning	Semi-objective	coherence, know
Case outcome prediction	Objective	impact, completion
Opposing counsel challenges	Objective	uncertainty, signal

Table 23: Legal evidence collectors and the vectors they ground.

Why Legal Grounding Is Stronger In software engineering, grounding relies on test results (pass/fail) and git metrics (lines changed)—useful but coarse signals. Legal work provides richer grounding because:

1. **Citations are binary verifiable**: A case citation either exists and says what the AI claims, or it doesn't. This provides KNOW grounding at quality 1.0 (objective) for every citation.

2. **Statutes are version-controlled**: Legislative databases track amendments with timestamps. Whether a statute is current as of a given date is deterministic—grounding CONTEXT and STATE with no ambiguity.

3. **Cost per verification is lower**: Checking a citation against a legal database is cheaper per claim than running a test suite against a codebase. More claims verified means higher grounded coverage per session.

This suggests that the grounded verification track, currently grounding 9 of 13 vectors in software engineering, could achieve even higher coverage in legal domains—potentially grounding 10+ of the 13 vectors with objective evidence.

Other Domains Beyond legal, other domains offer varying levels of grounding potential:

- Creative tasks (writing, design)—low grounding potential, quality is subjective
- Conversational agents—moderate, engagement assessable via user behavior metrics
- Scientific reasoning—high, claims verifiable against literature and data
- Embodied AI—high, physical actions produce measurable outcomes

7.7.9 Verification Schema Evolution: From Stochastic to Deterministic

A fundamental distinction underlies the grounded verification architecture: the AI's work and self-assessment are *stochastic inputs*—probabilistic, shaped by the model's epistemic state—while the verification logic is *deterministic output*—defined rules that produce pass/fail signals independent of the AI.

The path to reducing human oversight is systematically expanding the deterministic verification surface. We identify three tiers of verification quality:

Tier	What decides what to check	Example	Trust
1. Fully automated	System (predefined)	Artifact ratios, pytest, git	Highest
2. Schema-driven	Domain template	"Check all citations"	High
3. AI-selected	AI judgment	"I should verify X"	Medium

Table 24: Verification tiers by who determines check coverage.

Tier 1 checks are already implemented in the POST-TEST phase. Tier 3 is inherently bounded by the AI's own epistemic state—blindspots in the work become blindspots in verification. The critical lever is Tier 2: **domain-specific verification schemas** that define what must be checked, independent of the AI's judgment.

Schema Evolution Through Trajectory Analysis Verification schemas need not be static. As epistemic trajectories accumulate, patterns emerge:

1. Blindspot detection (via trajectory analysis) identifies recurring gaps in AI output
2. Each detected blindspot becomes a candidate for a new deterministic check
3. The check is added to the domain verification schema
4. Future sessions apply the check automatically (Tier 2, not Tier 3)
5. The stochastic surface area—where human verification is still needed—shrinks

This creates a measurable quantity: the **epistemic entropy** of a domain's verification state, defined as the gap between what the verification schema covers and what the full output space requires. As schemas expand through trajectory iteration, entropy decreases and the need for human verification diminishes proportionally.

Open Questions Several questions remain for this research direction:

- **Entropy formalization**: How to formally measure the entropy state of an epistemic trajectory—as a ratio, a distribution, or requiring domain-specific formalization?
- **Convergence rate**: How many trajectory iterations are needed to achieve target coverage in a given domain? Is there a diminishing returns curve?
- **Cold-start problem**: How to bootstrap verification schemas for new domains with no trajectory history. Can cross-domain transfer help (e.g., legal citation patterns informing medical literature verification)?
- **Schema completeness**: Can we prove that a verification schema is "complete" for a given domain, or is the output space inherently open-ended?

This direction connects the earned agency model (Section 6) to a concrete mechanism: autonomy increases not because we trust the AI's self-report, but because the deterministic verification layer expands to cover more of the output space. Trust follows coverage, not confidence.

7.7.10 Alignment Applications

The ENGAGEMENT vector's near-zero delta suggests potential alignment applications:

- Can engagement patterns distinguish aligned from misaligned interactions?
- Does low engagement correlate with adversarial or confused users?
- Can epistemic gating prevent harmful outputs by detecting low-engagement contexts?
- How do jailbreak attempts manifest in epistemic vectors?

DENSITY as Safety Signal Preliminary observations suggest extreme DENSITY values (approaching 1.0) correlate with AI system freezes or halts. This could indicate:

- **Cognitive overload**: Information packing exceeds processing capacity
- **Natural circuit breaker**: A safety mechanism preventing action under information saturation
- **Adversarial signature**: Prompt injection attacks often maximize information density to overwhelm context windows

The relationship between information density and system stability merits investigation. If high DENSITY reliably predicts system failure or adversarial input, it could serve as an early-warning signal for epistemic gating.

7.7.11 Real-Time and Streaming Applications

Production deployment may require lower-latency epistemic assessment:

- Streaming epistemic updates during generation
- Latency-aware gating that balances thoroughness with responsiveness
- Incremental calibration without full CASCADE cycles
- Edge deployment with limited computational resources

7.8 Summary

The epistemic vector framework and its empirical validation suggest that AI self-assessment is meaningful, measurable, and practically useful. The learning delta demonstrates that effective AI capability—model weights plus externalized epistemic scaffolding—genuinely changes during task engagement, supporting the noetic-before-praxic principle.

The limitations are real: grounded verification coverage varies by vector (IMPACT less grounded than KNOW), model concentration, domain specificity. The dual-track calibration system partially addresses the self-referential measurement concern, but full external validation remains incomplete. The framework provides a foundation for systematic investigation of AI epistemic state. We release all data for replication and extension.

8 Conclusion

8.1 Summary of Contributions

This paper presents Empirica, a framework for AI epistemic self-assessment, and validates it with substantial empirical evidence.

Conceptual contribution: We articulate the *noetic-before-praxic* principle—AI systems must investigate before acting—and argue that treating AI as an execution layer is a category error that produces systematic failures.

Theoretical contribution: We propose a 13-dimensional epistemic vector representation capturing what AI systems know, what they can do, and how ready they are to act. This constitutes a computational epistemology applicable to any information-processing system managing its own knowledge state.

Empirical contribution: We present 1,190,265 evidence observations from 792 production sessions demonstrating:

- Unified confidence growth across all 13 vectors during task execution
- Mean learning delta of +0.162 across 12 capability vectors (+0.114 for KNOW) with 83.3% of learning pairs showing improvement
- Calibration convergence: 107× variance reduction as evidence accumulates
- Systematic patterns supporting functional validity of self-assessment

Practical contribution: We release Empirica as open-source infrastructure for epistemic gating, dual-track calibration, and the 4-phase CASCADE workflow (PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST). The framework implements a seven-layer artifact taxonomy (noetic artifacts, epistemic intent, procedural knowledge, praxic artifacts, epistemic state, grounded evidence, context), a four-layer memory architecture (HOT/WARM/SEARCH/-COLD), and a learning pipeline where seven interlocking mechanisms produce compound capability growth across sessions—without modifying model weights. The epistemic intent layer tracks assumptions with urgency decay and decisions with provenance. The lesson system accumulates procedural knowledge subject to a cognitive immune system that decays stale procedures when contradicted by new findings.

8.2 The Core Claim

The central claim is simple: **AI systems that investigate before acting outperform AI systems that act without investigating**.

The evidence is in the delta. Effective AI capability—model weights plus externalized epistemic scaffolding—at task-end exceeds effective capability at task-start, across all measured dimensions, because investigation produces learning that the scaffolding captures and compounds. The model's weights remain static; what grows is the structured knowledge infrastructure around it. The overconfidence trap—acting on insufficient knowledge—is not just a failure mode to avoid; it is a learning opportunity foregone.

8.3 Implications

For AI practitioners: The execution-layer model is insufficient for complex tasks. Design AI systems as learning systems with epistemic gates that enforce investigation.

For AI researchers: Self-assessment is measurable and meaningful. The systematic patterns we observe invite replication and extension.

For AI safety: Epistemic gating offers a complementary approach to content filtering—preventing action when knowledge is insufficient rather than filtering harmful outputs after generation. The earned agency model provides a path to safe autonomous AI where trust is proportional to demonstrated calibration accuracy, domain-specific, and revocable.

8.4 The Delta

We close with the finding that motivated this work:

Phase	**Mean Assessment**
PREFLIGHT	0.647
POSTFLIGHT	0.809
Delta	**+0.162**
Improvement Rate	**83.3%**

Table 25: The learning delta: capability growth through investigation.

This is the learning that happens when AI systems are allowed—and required—to investigate before acting. It is the effective capability growth that execution-layer deployments forfeit. It is the empirical signature of noetic-before-praxic.

The delta is real. The learning—externalized through structured epistemic artifacts and compounded through persistent memory—is real. The framework calibrates: variance drops

107× as evidence accumulates, and two independent measurement tracks (self-referential and grounded) converge. The framework works.

Data Availability: Full dataset (1,190,265 evidence observations across 792 sessions) available at Zenodo: https://doi.org/10.5281/zenodo.18237263. Empirica framework and analysis code: https://github.com/Nubaeon/empirica.

Acknowledgments: This paper was developed in collaboration with Claude (Anthropic), which contributed to drafting, analysis framing, and iterative refinement across the research process.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. Does the whole exceed its parts? The effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2021.

Tristan Bricken and Anthropic. On the biology of a large language model: Emergent introspective awareness. *Transformer Circuits Thread*, 2025. https://transformer-circuits.pub/2025/introspection/index.html.

Stephen M Fleming and Raymond J Dolan. The neural basis of metacognitive ability. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 367(1594):1338–1349, 2012.

Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning*, pages 1050–1059. PMLR, 2016.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664*, 2023.

Vivian Lai, Chacha Chen, Q Vera Liao, Alison Smith-Renner, and Chenhao Tan. Towards a science of human-AI decision making: A survey of empirical studies. *arXiv preprint arXiv:2112.11471*, 2021.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Mark Steyvers and Garrett Peters. Metacognition in large language models: Current landscape and open questions. *Current Directions in Psychological Science*, 2025. doi: 10.1177/09637214251391158.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023b.